

Chương 2a - Mệnh đề Boolean

Mục lục

1	Logic	2
2	Logic Mệnh đề	3
2.1	Mệnh đề	3
2.2	Toán tử logic	4
2.3	Quy tắc ưu tiên	5
2.4	Tương đương logic	5
2.5	Thêm các toán tử logic	7
2.6	Phép kéo theo trong ngôn ngữ tự nhiên	7
2.7	Các dạng khác của phép kéo theo	8
2.8	Hoặc loại trừ	9
2.9	Toán tử phổ dụng	9
2.10	Phân loại mệnh đề	10
3	Đại số Bool	12
3.1	Cơ sở của Đại số Bool	13
3.2	Luật thay thế	15
3.3	Các phép đơn giản hóa	16
3.4	Thêm các quy tắc của đại số Bool	17
4	Ứng dụng: Mạch Logic	20
4.1	Cổng logic	21
4.2	Kết hợp cổng để tạo mạch	22
4.3	Từ mạch đến mệnh đề	22
4.4	Dạng chuẩn tắc tuyển	23
4.5	Cộng nhị phân	25

1 Logic

Theo một nghĩa nào đó, chúng ta biết nhiều hơn những gì chúng ta nhận ra, bởi vì mọi thứ chúng ta biết đều có các hệ quả—các hệ quả *logic*—được suy ra một cách tự động. Nếu bạn biết rằng tất cả con người đều phải chết, và bạn biết rằng Socrates là con người, thì theo một nghĩa nào đó bạn biết rằng Socrates phải chết, dù bạn có từng xem xét hoặc muốn xem xét sự thật đó hay không. Đây là một ví dụ về **suy diễn logic**: từ các **tiền đề** rằng “Tất cả con người đều phải chết” và “Socrates là con người”, **kết luận** rằng “Socrates phải chết” có thể được suy ra bằng logic.

Info

Socrates là một triết gia Hy Lạp có số phận khá bất hạnh. Trong một trong những lập luận toán học nổi tiếng nhất—lập luận được phác thảo ở trên—ông là người đầu tiên được định sẵn phải chết. Lịch sử từ đó đã dạy chúng ta rằng chúng ta đã đúng khi Socrates qua đời sau một cuộc đời dài (71 tuổi), bị kết án tử hình vì tội làm hư hỏng tâm trí của thanh niên Athens. Môn đệ của ông là Plato đã viết nhiều đối thoại Socratic cho ta cái nhìn sâu sắc về triết học của Socrates, thường được tóm tắt là: “Tôi biết rằng tôi không biết gì cả”. Lời cuối cùng của người phải chết này (theo Plato) là: “Crito, chúng ta nợ Asclepius một con gà trống: hãy trả nó và đừng quên.” Số phận con gà thì không ai biết...

Nguồn: en.wikipedia.org/wiki/Socrates.

Suy diễn logic là một dạng tính toán. Bằng cách áp dụng các quy tắc logic cho một tập tiền đề cho trước, các kết luận suy ra từ các tiền đề đó có thể được tạo ra một cách tự động. Quá trình tính toán này chẳng hạn có thể được thực hiện bởi một máy tính. Một khi bạn biết các tiền đề, hoặc sẵn sàng chấp nhận chúng vì mục đích tranh luận, bạn bị logic *buộc* phải chấp nhận các kết luận. Tuy nhiên, nói rằng bạn ‘biết’ các kết luận đó sẽ gây hiểu lầm. Vấn đề là có quá nhiều kết luận (vô hạn), và nhìn chung, hầu hết chúng không đặc biệt thú vị. Cho đến khi bạn thực sự thực hiện phép suy diễn, bạn không thực sự biết kết luận, và việc biết nên đi theo chuỗi suy diễn nào trong các chuỗi có thể không phải dễ dàng. *Nghệ thuật* của logic là tìm ra một kết luận thú vị và một chuỗi suy diễn logic dẫn từ các tiền đề đến kết luận đó. Việc kiểm tra rằng các suy diễn là hợp lệ là mặt cơ học, tính toán của logic.

Info

Sau này trong *Reasoning & Logic*, bạn sẽ thấy một số kỹ thuật tính toán tự động có thể giúp chúng ta kiểm tra các suy diễn. Chúng tôi không đề cập đến những kỹ thuật này trong ấn bản này. Thực tế, có các *trợ lý chứng minh tự động* thậm chí có thể giúp chúng ta tìm các kết luận thú vị. Một trong những trợ lý nổi tiếng nhất được gọi là *Coq*, một cái tên có lẽ được lấy cảm hứng từ con gà của Socrates.

Chương này chủ yếu nói về cơ chế của logic. Chúng ta sẽ nghiên cứu logic như một nhánh của toán học, với các ký hiệu, công thức và quy tắc tính toán riêng. Mục tiêu của bạn là học các quy tắc logic, hiểu tại sao chúng hợp lệ, và phát triển kỹ năng áp dụng chúng. Như với bất kỳ nhánh toán học nào, có một vẻ đẹp nhất định trong các ký hiệu và công thức. Nhưng chính các ứng dụng mới mang lại sức sống cho chủ đề này đối với hầu hết mọi người. Tất nhiên, chúng ta sẽ đề cập đến một số ứng dụng trong quá trình

học. Tuy nhiên, theo một nghĩa nào đó, các ứng dụng thực sự của logic bao gồm phần lớn khoa học máy tính và toán học.

Trong số các yếu tố cơ bản của tư duy, và do đó của logic, là các mệnh đề. Một **mệnh đề** là một phát biểu có một giá trị chân lý: nó đúng hoặc sai. “Delft là một thành phố” và “ $2 + 2 = 42$ ” là các mệnh đề. Trong phần đầu của chương này, chúng ta sẽ nghiên cứu **logic mệnh đề**, xem xét các mệnh đề và cách chúng có thể được kết hợp và thao tác. Nhánh logic này có ứng dụng đáng ngạc nhiên trong việc thiết kế các mạch điện tử tạo nên máy tính. Điều này liên hệ chặt chẽ với logic số và logic Bool mà bạn sẽ học trong môn *Computer Organisation*.

Logic trở nên thú vị hơn khi chúng ta xem xét cấu trúc bên trong của các mệnh đề. Trong tiếng Anh, một mệnh đề được diễn đạt dưới dạng một câu, và như bạn biết từ việc học ngữ pháp, câu có các thành phần. Một câu đơn giản như “Delft is a city” có một **chủ ngữ** và một **vị ngữ**. Câu nói điều gì đó về chủ ngữ của nó. Chủ ngữ của “Delft is a city” là Delft. Câu nói điều gì đó về Delft. Điều mà câu nói về chủ ngữ là vị ngữ. Trong ví dụ, vị ngữ là cụm từ ‘is a city’ (là một thành phố).

Khi chúng ta bắt đầu làm việc với các vị ngữ, chúng ta có thể tạo ra các mệnh đề sử dụng các **lượng từ** như ‘tất cả’, ‘một số’ và ‘không’. Ví dụ, khi làm việc với vị ngữ ‘có một trường đại học’, chúng ta có thể chuyển từ các mệnh đề đơn giản như “Delft có một trường đại học” sang “Tất cả các thành phố đều có trường đại học” hoặc “Không thành phố nào có trường đại học” hoặc sang phát biểu thực tế hơn “Một số thành phố có trường đại học”.

Suy diễn logic thường xử lý các phát biểu có lượng từ, như ví dụ cơ bản về sự phải chết của con người mà chúng ta đã bắt đầu chương này. Suy diễn logic sẽ là một chủ đề chính của chương này; và dưới tên gọi *chứng minh*, nó sẽ là chủ đề của chương tiếp theo và là một công cụ quan trọng cho phần còn lại của cuốn sách này cũng như chương trình khoa học máy tính của bạn.

2 Logic Mệnh đề

Con người sử dụng **ngôn ngữ tự nhiên** khi nói, chẳng hạn như tiếng Hà Lan, tiếng Anh hay tiếng Việt. Ngôn ngữ tự nhiên mơ hồ và thường không chính xác. Để bắt đầu mô hình hóa chúng, trước tiên chúng ta xem xét **logic mệnh đề**. Dạng logic này có lẽ dễ làm việc nhất, nhưng cũng có sức biểu đạt hạn chế. Tuy nhiên, ngay cả với dạng này, chúng ta đã có thể đóng gói nhiều lập luận và phục vụ một số ứng dụng, ví dụ logic số trong thiết kế chip.

2.1 Mệnh đề

Một mệnh đề là một phát biểu đúng hoặc sai. Trong logic mệnh đề, chúng ta chỉ lập luận về các mệnh đề và xem chúng ta có thể làm gì với chúng. Vì đây là toán học, chúng ta cần có khả năng nói về các mệnh đề mà không nói cụ thể chúng ta đang nói về mệnh đề nào, vì vậy chúng ta sử dụng các tên ký hiệu để đại diện cho chúng. Chúng ta sẽ luôn sử dụng các chữ cái thường như p , q và r để biểu diễn các mệnh đề. Một chữ cái được sử dụng theo cách này được gọi là **biến mệnh đề**. Hãy nhớ rằng khi chúng ta nói “Cho p là một mệnh đề”, chúng ta có nghĩa là “Trong phần còn lại của cuộc thảo luận này, cho ký hiệu p đại diện cho một phát biểu cụ thể nào đó, đúng hoặc sai (mặc dù lúc này chúng ta không đưa ra giả định nào về nó là cái nào).” Cuộc thảo luận có **tính tổng quát toán**

học vì p có thể đại diện cho bất kỳ phát biểu nào, và cuộc thảo luận sẽ hợp lệ bất kể phát biểu nào mà nó đại diện.

Note

Biến mệnh đề hơi giống các biến trong ngôn ngữ lập trình như Java. Một biến Java cơ bản như `int x` có thể nhận bất kỳ giá trị nguyên nào. Có ‘một chút’ tương đồng giữa hai khái niệm biến—đừng đẩy phép so sánh này đi quá xa ở giai đoạn học tập này!

2.2 Toán tử logic

Điều chúng ta làm với các mệnh đề là kết hợp chúng bằng các **toán tử logic**, cũng được gọi là **liên kết logic**. Một toán tử logic có thể được áp dụng cho một hoặc nhiều mệnh đề để tạo ra một mệnh đề mới. Giá trị chân lý của mệnh đề mới được xác định hoàn toàn bởi toán tử và bởi các giá trị chân lý của các mệnh đề mà nó được áp dụng.¹ Trong tiếng Anh, các toán tử logic được biểu diễn bằng các từ như ‘và’, ‘hoặc’ và ‘không’. Ví dụ, mệnh đề “Tôi muốn rời đi và tôi đã rời đi” được hình thành từ hai mệnh đề đơn giản hơn nối bằng từ ‘và’. Thêm từ ‘không’ vào mệnh đề “Tôi đã rời đi” cho ra “Tôi đã không rời đi” (sau một chút điều chỉnh ngữ pháp cần thiết).

Nhưng tiếng Anh (hay bất kỳ ngôn ngữ tự nhiên nào) quá phong phú cho logic toán học. Khi bạn đọc câu “Tôi muốn rời đi và tôi đã rời đi”, bạn có thể thấy hàm ý nhân quả: tôi đã rời đi *vì* tôi muốn rời đi. Suy luận này không theo từ sự kết hợp logic của các giá trị chân lý của hai mệnh đề “Tôi muốn rời đi” và “Tôi đã rời đi”. Hoặc xem xét mệnh đề “Tôi muốn rời đi nhưng tôi đã không rời đi”. Ở đây, từ ‘nhưng’ có cùng ý nghĩa logic như từ ‘và’, nhưng hàm ý rất khác. Vì vậy, trong logic toán học, chúng ta sử dụng các *ký hiệu* để biểu diễn các toán tử logic. Các ký hiệu này không mang bất kỳ hàm ý nào ngoài ý nghĩa logic đã định nghĩa của chúng. Các toán tử logic tương ứng với các từ tiếng Anh ‘and’ (và), ‘or’ (hoặc) và ‘not’ (không) là $\wedge$, $\vee$ và $\neg$.²

Định nghĩa 1. Cho p và q là các mệnh đề. Khi đó $p \vee q$, $p \wedge q$, và $\neg p$ là các mệnh đề, với giá trị chân lý được cho bởi các quy tắc:

- $p \wedge q$ đúng khi cả p đúng và q đúng, và không đúng trong trường hợp nào khác.
- $p \vee q$ đúng khi p đúng, hoặc q đúng, hoặc cả p và q đều đúng, và không đúng trong trường hợp nào khác.
- $\neg p$ đúng khi p sai, và không đúng trong trường hợp nào khác.

Các toán tử $\wedge$, $\vee$ và $\neg$ được gọi lần lượt là **hội** (conjunction), **tuyển** (disjunction) và **phủ định** (negation). (Lưu ý rằng $p \wedge q$ được đọc là ‘ p và q ’, $p \vee q$ được đọc là ‘ p hoặc q ’, và $\neg p$ được đọc là ‘không p ’.)

¹Không phải lúc nào giá trị chân lý của một câu cũng có thể được xác định từ giá trị chân lý của các thành phần của nó. Ví dụ, nếu p là một mệnh đề, thì “Johan Cruyff tin rằng p ” cũng là một mệnh đề, vì vậy “Cruyff tin rằng” là một dạng toán tử. Tuy nhiên, nó không được tính là toán tử logic bởi vì chỉ từ việc biết p đúng hay sai, chúng ta không có thông tin gì về việc “Johan Cruyff tin rằng p ” đúng hay sai.

²Các sách giáo khoa khác có thể sử dụng các ký hiệu khác để biểu diễn phủ định. Chẳng hạn dấu gạch ngang trên biến ($\bar{x}$) hoặc ký hiệu $\sim$ ($\sim x$). Trong đại số Bool (và do đó trong môn *Computer Organisation* của bạn), bạn cũng thường thấy ký hiệu $+$ để biểu diễn ‘hoặc’ và ký hiệu $\cdot$ (chấm) để biểu diễn ‘và’.

Note

Xét phát biểu “Tôi là sinh viên CSE hoặc tôi không phải là sinh viên TPM.” Cho p có nghĩa “Tôi là sinh viên CSE” và q có nghĩa “Tôi là sinh viên TPM”, bạn có thể viết phát biểu này là $p \vee \neg q$.

2.3 Quy tắc ưu tiên

Các toán tử này có thể được sử dụng trong các biểu thức phức tạp hơn, như $p \wedge \neg q$ hoặc $(p \vee q) \wedge (q \vee r)$. Một mệnh đề được tạo từ các mệnh đề đơn giản hơn và các toán tử logic được gọi là **mệnh đề hợp**. Giống như trong toán học, dấu ngoặc có thể được sử dụng trong các biểu thức hợp để chỉ thứ tự các toán tử được đánh giá. Khi không có dấu ngoặc, thứ tự đánh giá được xác định bởi **quy tắc ưu tiên**. Đối với các toán tử logic đã định nghĩa ở trên, các quy tắc là: $\neg$ có ưu tiên cao hơn $\wedge$, và $\wedge$ có ưu tiên cao hơn $\vee$. Điều này có nghĩa là khi không có dấu ngoặc, mọi toán tử $\neg$ được đánh giá trước, sau đó là mọi toán tử $\wedge$, rồi đến mọi toán tử $\vee$.

Ví dụ, biểu thức $\neg p \vee q \wedge r$ tương đương với biểu thức $(\neg p) \vee (q \wedge r)$, trong khi $p \vee q \wedge q \vee r$ tương đương với $p \vee (q \wedge q) \vee r$.

Điều này vẫn để ngỏ câu hỏi toán tử $\wedge$ nào trong biểu thức $p \wedge q \wedge r$ được đánh giá trước. Điều này được giải quyết bởi quy tắc sau: Khi nhiều toán tử cùng mức ưu tiên xuất hiện mà không có dấu ngoặc, chúng được đánh giá từ trái sang phải. Do đó, biểu thức $p \wedge q \wedge r$ tương đương với $(p \wedge q) \wedge r$ chứ không phải $p \wedge (q \wedge r)$. Trong trường hợp cụ thể này, thực tế không quan trọng toán tử $\wedge$ nào được đánh giá trước, vì hai mệnh đề hợp $(p \wedge q) \wedge r$ và $p \wedge (q \wedge r)$ luôn có cùng giá trị, bất kể các giá trị logic của các mệnh đề thành phần p , q và r là gì. Chúng ta nói rằng $\wedge$ là một phép toán **kết hợp** (associative). Chúng ta sẽ tìm hiểu thêm về tính kết hợp và các tính chất khác của phép toán trong phần tiếp theo.

Cảnh báo

Tuy nhiên, trong thực tế bạn nên **luôn luôn** thêm dấu ngoặc ở những nơi có thể phát sinh sự mơ hồ. Thực tế, một số sách giáo khoa thậm chí thêm dấu ngoặc cho cả các toán tử đơn lẻ, ví dụ viết $(p \wedge q)$ thay vì $p \wedge q$. Mặc dù trong môn học này chúng tôi không yêu cầu dấu ngoặc quanh các toán tử đơn lẻ, chúng ta không bao giờ cần phải dùng đến các quy tắc ưu tiên nêu trên. Dấu ngoặc của bạn nên làm rõ thứ tự các phép toán!

Mọi mệnh đề hợp đều có một **liên kết chính**. Liên kết chính là liên kết được đánh giá cuối cùng, theo các quy tắc ưu tiên và dấu ngoặc. Không nên có sự mơ hồ về đâu là liên kết chính trong một mệnh đề hợp.

2.4 Tương đương logic

Giả sử chúng ta muốn xác minh rằng, thực sự, $(p \wedge q) \wedge r$ và $p \wedge (q \wedge r)$ luôn có cùng giá trị. Để làm vậy, chúng ta phải xem xét tất cả các tổ hợp giá trị có thể của p , q và r , và kiểm tra rằng với mọi tổ hợp như vậy, hai biểu thức hợp thực sự có cùng giá trị. Sẽ tiện lợi khi tổ chức phép tính này vào một bảng chân trị. **Bảng chân trị** là một bảng cho thấy giá trị của một hoặc nhiều mệnh đề hợp cho mỗi tổ hợp giá trị có thể của các biến mệnh đề chứa trong chúng. Chúng ta gọi mỗi tổ hợp như vậy là một **trình huống**. Hình 1

p	q	r	$p \wedge q$	$q \wedge r$	$(p \wedge q) \wedge r$	$p \wedge (q \wedge r)$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

Hình 1: Bảng chân trị minh họa sự tương đương logic của $(p \wedge q) \wedge r$ và $p \wedge (q \wedge r)$. Việc hai cột cuối của bảng giống hệt nhau cho thấy hai biểu thức này có cùng giá trị cho tất cả tám tổ hợp giá trị có thể của p , q và r .

là một bảng chân trị so sánh giá trị của $(p \wedge q) \wedge r$ với giá trị của $p \wedge (q \wedge r)$ cho tất cả các giá trị có thể của p , q và r . Có tám hàng trong bảng vì có đúng tám cách khác nhau để gán giá trị chân lý cho p , q và r .³ Trong bảng này, chúng ta thấy hai cột cuối, biểu diễn giá trị của $(p \wedge q) \wedge r$ và $p \wedge (q \wedge r)$, là giống hệt nhau.

Note

Bạn có thể viết các hàng trong bảng chân trị theo bất kỳ thứ tự nào bạn muốn. Chúng tôi gợi ý bạn viết chúng theo thứ tự sắp xếp, như trong Hình 1. Điều này giúp bạn có hệ thống khi viết bảng. Nó cũng giúp chúng tôi cung cấp phản hồi cho bài làm của bạn!

Tổng quát hơn, chúng ta nói rằng hai mệnh đề hợp là **tương đương logic** nếu chúng luôn có cùng giá trị, bất kể giá trị chân lý nào được gán cho các biến mệnh đề chứa trong chúng. Nếu số biến mệnh đề nhỏ, việc sử dụng bảng chân trị để kiểm tra hai mệnh đề có tương đương logic hay không là dễ dàng.

Note

Khi viết một đoạn mã, bạn thường sẽ để mã của mình ra quyết định. Ví dụ trong một đoạn mã Java—chẳng hạn trong môn *Object-Oriented Programming* của bạn—bạn có thể gặp một câu lệnh `if` để kiểm tra xem người dùng đã nhập đúng loại dữ liệu hay chưa. Vì đầu vào bạn mong đợi có thể khá phức tạp, câu lệnh `if` là một tổ hợp phức tạp của nhiều kiểm tra đơn giản nối với nhau bằng `&&` và `||`. Sau khi xem lại mã, bạn tin rằng nó có thể được đơn giản hóa thành một biểu thức nhỏ hơn nhiều. Sử dụng bảng chân trị, bạn có thể chứng minh rằng phiên bản đơn giản hóa của bạn tương đương với bản gốc.

³Tổng quát, nếu có n biến, thì có 2^n cách khác nhau để gán giá trị chân lý cho các biến, tức là 2^n tình huống. Điều này có thể rõ ràng với bạn nếu bạn thử nghĩ ra một sơ đồ liệt kê có hệ thống tất cả các tập giá trị có thể. Vì điều này chưa nên làm bạn hài lòng, bạn sẽ tìm thấy một chứng minh chặt chẽ về sự thật này ở phần sau trong chương.

2.5 Thêm các toán tử logic

Có các toán tử logic khác ngoài $\wedge$, $\vee$ và $\neg$. Chúng ta sẽ xem xét **toán tử điều kiện** (kéo theo), $\rightarrow$, **toán tử song điều kiện** (tương đương), $\leftrightarrow$, và **toán tử hoặc loại trừ**, $\oplus$.⁴ Các toán tử này có thể được định nghĩa hoàn toàn bằng bảng chân trị cho thấy giá trị của chúng với bốn tổ hợp giá trị chân lý có thể của p và q .

Định nghĩa 2. Với mọi mệnh đề p và q , ta định nghĩa các mệnh đề $p \rightarrow q$, $p \leftrightarrow q$, và $p \oplus q$ theo bảng chân trị:

p	q	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
0	0	1	1	0
0	1	1	0	1
1	0	0	0	1
1	1	1	1	0

Info

Khi các toán tử này được sử dụng trong các biểu thức, nếu không có dấu ngoặc để chỉ thứ tự đánh giá, chúng ta sử dụng các quy tắc ưu tiên sau: Toán tử hoặc loại trừ, $\oplus$, có cùng mức ưu tiên với $\vee$. Toán tử điều kiện, $\rightarrow$, có ưu tiên thấp hơn $\wedge$, $\vee$, $\neg$ và $\oplus$, và do đó được đánh giá sau chúng. Cuối cùng, toán tử song điều kiện, $\leftrightarrow$, có ưu tiên thấp nhất và do đó được đánh giá cuối cùng. Ví dụ, biểu thức $p \rightarrow q \wedge r \leftrightarrow \neg p \oplus s$ được đánh giá như thể nó được viết là $(p \rightarrow (q \wedge r)) \leftrightarrow ((\neg p) \oplus s)$. Nhưng một lần nữa bạn nên luôn thêm dấu ngoặc!

Để làm việc hiệu quả với các toán tử logic, bạn cần biết thêm về ý nghĩa của chúng và cách chúng liên hệ với các biểu đạt trong ngôn ngữ tự nhiên. Với mục đích đó, trước tiên chúng ta xem xét toán tử điều kiện chi tiết hơn trong phần tiếp theo.

2.6 Phép kéo theo trong ngôn ngữ tự nhiên

Mệnh đề $p \rightarrow q$ được gọi là **phép kéo theo** hay **mệnh đề điều kiện**. Nó thường được đọc là ‘ p kéo theo q ’. Trong một phép kéo theo như vậy, p và q cũng có tên riêng. p được gọi là **giả thiết** hay **tiền kiện** (antecedent) và q được gọi là **kết luận** hay **hậu kiện** (consequent).

Hơn nữa, chúng ta nói rằng nếu phép kéo theo $p \rightarrow q$ đúng, thì p là **điều kiện đủ** cho q . Nghĩa là nếu p đúng thì đó là đủ để q cũng đúng. Ngược lại, chúng ta nói q là **điều kiện cần** cho p . Nếu q không đúng, thì không thể p đúng được. Nghĩa là nếu q sai, thì p cũng phải sai.

Trong tiếng Anh, $p \rightarrow q$ thường được diễn đạt là ‘nếu p thì q ’. Ví dụ, nếu p biểu diễn mệnh đề “Karel Luyben là Hiệu trưởng Đại học TU Delft” và q biểu diễn “Prometheus được các vị thần ban phước”, thì $p \rightarrow q$ có thể được diễn đạt là “Nếu Karel Luyben là Hiệu trưởng Đại học TU Delft, thì Prometheus được các vị thần ban phước.” Trong ví dụ này, p sai và q cũng sai. Kiểm tra định nghĩa của $p \rightarrow q$, chúng ta thấy $p \rightarrow q$ là một phát biểu đúng. Hầu hết mọi người đồng ý với điều này, mặc dù nó không hiển nhiên ngay lập tức.

⁴Lưu ý rằng các ký hiệu cho các phép toán này cũng khác nhau giữa các sách. Trong khi $\rightarrow$ khá chuẩn, $\leftrightarrow$ đôi khi được biểu diễn bằng $\equiv$ hoặc $\Leftrightarrow$. Ký hiệu cho toán tử hoặc loại trừ thậm chí còn ít chuẩn hóa hơn, nhưng toán tử đó nhìn chung không quan trọng bằng các toán tử khác.

Info

Chữ ‘T’ trong logo **TU**Delft mang một ngọn lửa cách điệu ở trên đỉnh, ám chỉ ngọn lửa mà Prometheus đã mang từ đỉnh Olympus xuống cho con người, trái ý Zeus. Vì điều này, Prometheus đôi khi được coi là kỹ sư đầu tiên, và ông là một biểu tượng quan trọng của trường đại học. Bức tượng đồng của ông đứng ở Mekelpark tại trung tâm khuôn viên.

Nguồn: en.wikipedia.org/wiki/Delft_University_of_Technology.

Đáng để xem xét một ví dụ tương tự chi tiết hơn. Giả sử tôi khẳng định rằng “Nếu Feyenoord là một đội bóng vĩ đại, thì tôi là Vua của Hà Lan”. Phát biểu này có dạng $m \rightarrow k$ trong đó m là mệnh đề “Feyenoord là một đội bóng vĩ đại” và k là mệnh đề “Tôi là vua của Hà Lan”. Rõ ràng tôi không phải là vua của Hà Lan, nên k sai. Vì k sai, cách duy nhất để $m \rightarrow k$ đúng là m cũng phải sai. (Kiểm tra định nghĩa của $\rightarrow$ trong bảng, nếu bạn chưa tin!) Vì vậy, bằng việc khẳng định $m \rightarrow k$, tôi thực sự đang khẳng định rằng Feyenoord *không phải* là một đội bóng vĩ đại.

Hoặc xem xét phát biểu “Nếu bữa tiệc vào thứ Ba, thì tôi sẽ có mặt.” Tôi đang muốn nói gì khi khẳng định phát biểu này? Tôi đang khẳng định rằng $p \rightarrow q$ đúng, trong đó p biểu diễn “Bữa tiệc vào thứ Ba” và q biểu diễn “Tôi sẽ ở bữa tiệc”. Giả sử p đúng, tức là bữa tiệc thực sự diễn ra vào thứ Ba. Kiểm tra định nghĩa của $\rightarrow$, chúng ta thấy rằng trong trường hợp duy nhất khi p đúng và $p \rightarrow q$ đúng, thì q cũng đúng. Vì vậy từ sự đúng của “Nếu bữa tiệc vào thứ Ba, thì tôi sẽ ở bữa tiệc” và “Bữa tiệc thực sự vào thứ Ba”, bạn có thể suy ra “Tôi sẽ ở bữa tiệc” cũng đúng. Nhưng giả sử, mặt khác, bữa tiệc thực sự vào thứ Tư. Khi đó p sai. Khi p sai và $p \rightarrow q$ đúng, định nghĩa của $p \rightarrow q$ cho phép q đúng hoặc sai. Vì vậy, trong trường hợp này, bạn không thể suy ra bất cứ điều gì về việc tôi có ở bữa tiệc hay không. Phát biểu “Nếu bữa tiệc vào thứ Ba, thì tôi sẽ có mặt” không khẳng định điều gì về những gì sẽ xảy ra nếu bữa tiệc vào ngày khác ngoài thứ Ba.

2.7 Các dạng khác của phép kéo theo

Phép kéo theo $\neg q \rightarrow \neg p$ được gọi là **phản đề** (contrapositive) của $p \rightarrow q$. Một phép kéo theo tương đương logic với phản đề của nó. Phản đề của “Nếu hôm nay là thứ Ba, thì chúng ta đang ở Bỉ” là “Nếu chúng ta không ở Bỉ, thì hôm nay không phải thứ Ba”. Hai câu này khẳng định cùng một điều.

Lưu ý rằng $p \rightarrow q$ *không* tương đương logic với $q \rightarrow p$. Phép kéo theo $q \rightarrow p$ được gọi là **mệnh đề đảo** (converse) của $p \rightarrow q$. Mệnh đề đảo của “Nếu hôm nay là thứ Ba, thì chúng ta đang ở Bỉ” là “Nếu chúng ta đang ở Bỉ, thì hôm nay là thứ Ba”. Lưu ý rằng có thể một trong hai phát biểu đúng trong khi phát biểu kia sai. Trong tiếng Anh, chúng ta có thể diễn đạt sự kiện cả hai phát biểu đều đúng bằng cách nói “Nếu hôm nay là thứ Ba, thì chúng ta đang ở Bỉ, *và ngược lại*”. Trong logic, điều này sẽ được diễn đạt bằng một mệnh đề có dạng $(p \rightarrow q) \wedge (q \rightarrow p)$.

Tương tự, $p \rightarrow q$ *không* tương đương logic với $\neg p \rightarrow \neg q$. Phép kéo theo $\neg p \rightarrow \neg q$ được gọi là **mệnh đề nghịch đảo** (inverse) của $p \rightarrow q$. Mặc dù sai lầm này thường gặp trong tiếng Anh, ví dụ người ta thường cho rằng khi tôi nói: “Nếu là buổi sáng, tôi uống cà phê”, tôi cũng có nghĩa là khi không phải buổi sáng tôi không uống cà phê. Nhưng phát biểu ban đầu của tôi không nói gì về những gì tôi làm khi không phải buổi sáng.

Toán tử song điều kiện có liên hệ chặt chẽ với toán tử điều kiện. Thực tế, $p \leftrightarrow q$ tương đương logic với $(p \rightarrow q) \wedge (q \rightarrow p)$. Mệnh đề $p \leftrightarrow q$ thường được đọc là ‘ p khi và

chỉ khi q '. (Phần ' p nếu q ' biểu diễn $q \rightarrow p$, trong khi ' p chỉ nếu q ' là cách khác để khẳng định $p \rightarrow q$.) Nó cũng có thể được diễn đạt là 'nếu p thì q , và ngược lại'. Đôi khi trong tiếng Anh, 'nếu... thì' được dùng khi ý thực sự là 'khi và chỉ khi'. Ví dụ, nếu một phụ huynh nói với con "Nếu con ngoan, Sinterklaas sẽ mang đồ chơi cho con", phụ huynh có lẽ thực sự muốn nói "Sinterklaas sẽ mang đồ chơi cho con khi và chỉ khi con ngoan". (Phụ huynh có lẽ sẽ không phản ứng tốt trước lời cầu xin hoàn toàn hợp lý của đứa trẻ "Nhưng bố/mẹ chưa bao giờ nói điều gì sẽ xảy ra nếu con không ngoan!")

2.8 Hoặc loại trừ

Cuối cùng, chúng ta chuyển sang toán tử hoặc loại trừ. Từ 'hoặc' trong tiếng Anh (hay tiếng Việt) thực sự có phần mơ hồ. Hai toán tử $\oplus$ và $\vee$ diễn đạt hai ý nghĩa có thể của từ này. Mệnh đề $p \vee q$ có thể được diễn đạt rõ ràng là " p hoặc q , hoặc cả hai", trong khi $p \oplus q$ có nghĩa " p hoặc q , nhưng không phải cả hai". Nếu thực đơn nói rằng bạn có thể chọn súp hoặc salad, điều đó không có nghĩa bạn có thể có cả hai. Trong trường hợp này, 'hoặc' là hoặc loại trừ. Mặt khác, trong "Bạn có nguy cơ mắc bệnh tim nếu bạn hút thuốc hoặc uống rượu", phép hoặc là bao hàm vì bạn chắc chắn không thoát được nếu bạn vừa hút thuốc vừa uống rượu. Trong khoa học máy tính lý thuyết và toán học, từ 'hoặc' luôn được hiểu theo nghĩa bao hàm của $p \vee q$.

2.9 Toán tử phổ dụng

Bất kỳ mệnh đề hợp nào sử dụng bất kỳ toán tử nào trong $\rightarrow$, $\leftrightarrow$ và $\oplus$ đều có thể được viết lại thành mệnh đề tương đương logic chỉ sử dụng $\wedge$, $\vee$ và $\neg$. Dễ dàng kiểm tra rằng $p \rightarrow q$ tương đương logic với $\neg p \vee q$. (Chỉ cần lập bảng chân trị cho $\neg p \vee q$.) Tương tự, $p \leftrightarrow q$ có thể được biểu diễn là $(\neg p \vee q) \wedge (\neg q \vee p)$. Vì vậy, theo nghĩa logic chặt chẽ, $\rightarrow$, $\leftrightarrow$ và $\oplus$ là không cần thiết. (Tuy nhiên, chúng hữu ích và quan trọng, và chúng ta sẽ không bỏ chúng.)

Thậm chí hơn thế nữa: theo nghĩa logic chặt chẽ, chúng ta có thể không cần toán tử hội $\wedge$. Dễ dàng kiểm tra rằng $p \wedge q$ tương đương logic với $\neg(\neg p \vee \neg q)$, vì vậy bất kỳ biểu thức nào sử dụng $\wedge$ đều có thể được viết lại chỉ dùng $\neg$ và $\vee$. Hoặc, chúng ta có thể không cần $\vee$ và viết mọi thứ bằng $\neg$ và $\wedge$. Chúng ta sẽ nghiên cứu một số quy tắc viết lại này chi tiết hơn trong Mục 3.

Chúng ta gọi một tập toán tử có thể biểu diễn tất cả các phép toán là **đầy đủ về mặt chức năng** (functionally complete). Chính thức hơn, ta phát biểu:

Định nghĩa 3. Một tập các toán tử logic là **đầy đủ về mặt chức năng** khi và chỉ khi mọi công thức trong logic mệnh đề đều có thể được viết lại sang dạng tương đương chỉ sử dụng các toán tử trong tập đó.

Xét chẳng hạn tập $\{\neg, \vee\}$. Như đã chỉ ra ở trên, các toán tử $\wedge$, $\rightarrow$ và $\leftrightarrow$ có thể được biểu diễn chỉ bằng các toán tử này. Thực tế, tất cả các phép toán có thể đều có thể được biểu diễn chỉ bằng $\{\neg, \vee\}$. Để chứng minh điều này, bạn sẽ chỉ ra trong một bài tập rằng mọi công thức trong logic mệnh đề có thể được biểu diễn bằng $\{\neg, \vee, \wedge, \rightarrow, \leftrightarrow\}$. Vì vậy bằng cách chỉ ra rằng chúng ta không cần $\wedge$, $\rightarrow$ và $\leftrightarrow$, chúng ta có thể chứng minh $\{\neg, \vee\}$ cũng đầy đủ về mặt chức năng.

2.10 Phân loại mệnh đề

Một số loại mệnh đề sẽ đóng vai trò đặc biệt trong công việc tiếp theo của chúng ta với logic. Cụ thể, chúng ta định nghĩa hằng đúng, mâu thuẫn và mệnh đề bất định như sau:

Định nghĩa 4. Một mệnh đề hợp được gọi là **hằng đúng** (tautology) khi và chỉ khi nó đúng với mọi tổ hợp giá trị chân lý có thể của các biến mệnh đề chứa trong nó. Một mệnh đề hợp được gọi là **mâu thuẫn** (contradiction) hay hằng sai khi và chỉ khi nó sai với mọi tổ hợp giá trị chân lý có thể của các biến mệnh đề chứa trong nó. Một mệnh đề hợp được gọi là **mệnh đề bất định** (contingency) khi và chỉ khi nó không phải hằng đúng cũng không phải mâu thuẫn.

Ví dụ, mệnh đề $((p \vee q) \wedge \neg q) \rightarrow p$ là một hằng đúng. Điều này có thể được kiểm tra bằng bảng chân trị:

p	q	$p \vee q$	$\neg q$	$(p \vee q) \wedge \neg q$	$((p \vee q) \wedge \neg q) \rightarrow p$
0	0	0	1	0	1
0	1	1	0	0	1
1	0	1	1	1	1
1	1	1	0	0	1

Việc tất cả các ô trong cột cuối đều là đúng cho ta biết biểu thức này là hằng đúng. Lưu ý rằng với bất kỳ mệnh đề hợp P nào, P là hằng đúng khi và chỉ khi $\neg P$ là mâu thuẫn. (Ở đây và sau này, chúng ta dùng chữ cái in hoa để biểu diễn các mệnh đề hợp. P đại diện cho bất kỳ công thức nào được tạo từ các mệnh đề đơn giản, biến mệnh đề và toán tử logic.)

Tương đương logic có thể được định nghĩa qua hằng đúng:

Định nghĩa 5. Hai mệnh đề hợp, P và Q , được gọi là **tương đương logic** khi và chỉ khi mệnh đề $P \leftrightarrow Q$ là một hằng đúng.

Khẳng định rằng P tương đương logic với Q sẽ được biểu diễn bằng ký hiệu ' $P \equiv Q$ '. Ví dụ, $(p \rightarrow q) \equiv (\neg p \vee q)$, và $p \oplus q \equiv (p \vee q) \wedge \neg(p \wedge q)$.

Info

Điều gì xảy ra nếu $P \rightarrow Q$ và P sai? Từ một tiền đề sai, chúng ta có thể suy ra bất kỳ kết luận nào (hãy kiểm tra bảng chân trị của $\rightarrow$). Vì vậy nếu k đại diện cho “Tôi là Vua của Hà Lan”, thì $k \rightarrow Q$ đúng với bất kỳ mệnh đề hợp Q nào. Bạn có thể thay thế bất cứ gì cho Q , và phép kéo theo $k \rightarrow Q$ sẽ đúng. Ví dụ, đây là một suy diễn hợp lệ về mặt logic: Nếu tôi là Vua Hà Lan, thì kỳ lân tồn tại. Xa hơn, từ một mâu thuẫn ta có thể suy ra bất kỳ kết luận nào. Điều này được gọi là **Nguyên lý Bùng nổ** (Principle of Explosion). (Không có kỳ lân nào bị hại trong việc giải thích nguyên lý này.)

Bài tập

Note

Hãy nhớ rằng lời giải cho một số bài tập bắt đầu ở trang 183. Các bài tập có lời giải được đánh dấu bằng ký hiệu dao găm ($\dagger$). Chúng tôi gợi ý bạn thử làm bài tập trước khi xem lời giải!

- †1. Cho ba bảng chân trị định nghĩa các toán tử logic $\wedge$, $\vee$ và $\neg$.
- †2. Một số mệnh đề hợp sau đây là hằng đúng, một số là mâu thuẫn, và một số không phải cả hai (tức là mệnh đề bất định). Trong mỗi trường hợp, hãy sử dụng bảng chân trị để quyết định mệnh đề thuộc loại nào:
- a) $(p \wedge (p \rightarrow q)) \rightarrow q$
 - b) $((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$
 - c) $p \wedge \neg p$
 - d) $(p \vee q) \rightarrow (p \wedge q)$
 - e) $p \vee \neg p$
 - f) $(p \wedge q) \rightarrow (p \vee q)$
- †3. Dùng bảng chân trị để chỉ ra rằng mỗi mệnh đề sau đây tương đương logic với $p \leftrightarrow q$.
- a) $(p \rightarrow q) \wedge (q \rightarrow p)$
 - b) $\neg p \leftrightarrow \neg q$
 - c) $(p \rightarrow q) \wedge (\neg p \rightarrow \neg q)$
 - d) $\neg(p \oplus q)$
- †4. $\rightarrow$ có phải là phép toán kết hợp không? Tức là, $(p \rightarrow q) \rightarrow r$ có tương đương logic với $p \rightarrow (q \rightarrow r)$ không?
- †5. Cho p biểu diễn mệnh đề “Bạn rời đi” và q biểu diễn mệnh đề “Tôi rời đi”. Biểu diễn các câu sau dưới dạng mệnh đề hợp sử dụng p và q , và chỉ ra rằng chúng tương đương logic:
- a) Hoặc bạn rời đi hoặc tôi rời đi. (Hoặc cả hai!)
 - b) Nếu bạn không rời đi, tôi sẽ rời đi.
- †6. Giả sử m biểu diễn mệnh đề “Trái Đất chuyển động”, c biểu diễn “Trái Đất là trung tâm vũ trụ”, và g biểu diễn “Galileo bị buộc tội oan”. Dịch mỗi mệnh đề hợp sau sang tiếng Việt:
- a) $\neg g \wedge c$
 - b) $m \rightarrow \neg c$
 - c) $m \leftrightarrow \neg c$
 - d) $(m \rightarrow g) \wedge (c \rightarrow \neg g)$
- †7. Cho mệnh đề đảo và phản đề của mỗi câu tiếng Anh sau:
- a) Nếu bạn ngoan, Sinterklaas sẽ mang đồ chơi cho bạn.
 - b) Nếu gói hàng nặng hơn một kilô, thì bạn cần thêm bưu phí.
 - c) Nếu tôi có sự lựa chọn, tôi không ăn bí ngòi.
- †8. Trong một bộ bài năm mươi hai lá tiêu chuẩn, có bao nhiêu lá bài mà phát biểu sau là đúng:

- a) “Lá bài này là con mười và lá bài này là cơ”?
 - b) “Lá bài này là con mười hoặc lá bài này là cơ”?
 - c) “Nếu lá bài này là con mười, thì lá bài này là cơ”?
 - d) “Lá bài này là con mười khi và chỉ khi lá bài này là cơ”?
- †9. Định nghĩa toán tử logic $\downarrow$ sao cho $p \downarrow q$ tương đương logic với $\neg(p \vee q)$. (Toán tử này thường được gọi là ‘NOR’, viết tắt của ‘not or’.) Chỉ ra rằng mỗi mệnh đề $\neg p$, $p \wedge q$, $p \vee q$, $p \rightarrow q$, $p \leftrightarrow q$ và $p \oplus q$ có thể được viết lại thành mệnh đề tương đương logic chỉ sử dụng $\downarrow$ làm toán tử duy nhất.
- †10. Để chứng minh $\{\neg, \vee\}$ đầy đủ về mặt chức năng, ta cần chỉ ra rằng mọi công thức trong logic mệnh đề có thể được biểu diễn ở dạng tương đương chỉ sử dụng $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$.
- a) Có bao nhiêu bảng chân trị phân biệt tồn tại cho các công thức chứa hai biến nguyên tử?
 - b) Tạo một hàm cho mỗi bảng chân trị có thể mà chỉ sử dụng 5 toán tử liệt kê ở trên.
 - c) Đưa ra một lập luận (không hình thức) giải thích tại sao điều này có nghĩa mọi công thức trong logic mệnh đề đều có thể được biểu diễn chỉ bằng năm toán tử này.

3 Đại số Bool

Cho đến nay chúng ta đã thảo luận cách viết và diễn giải các mệnh đề. Phần này bàn về việc *biến đổi* chúng. Để làm điều này, chúng ta cần đại số. Đại số thông thường, loại được dạy ở trường phổ thông, là về việc biến đổi các số, các biến đại diện cho số, và các toán tử như $+$ và $\times$ áp dụng cho số. Bây giờ, chúng ta cần một đại số áp dụng cho các giá trị logic, biến mệnh đề và toán tử logic. Người đầu tiên nghĩ về logic dưới góc độ đại số là nhà toán học George Boole, người đã giới thiệu ý tưởng này trong một cuốn sách xuất bản năm 1854. Đại số logic ngày nay được gọi là **đại số Bool** (Boolean algebra) để vinh danh ông.

Info

George Boole (1815–1864) là một nhà toán học, triết gia và nhà logic học người Anh phần lớn tự học, phần lớn sự nghiệp ngắn ngủi của ông được dành tại vị trí giáo sư toán học đầu tiên tại Queen's College, Cork ở Ireland. Ông làm việc trong lĩnh vực phương trình vi phân và logic đại số, và được biết đến nhiều nhất với tư cách là tác giả của *The Laws of Thought* (1854). Trong số sinh viên TU Delft, ông được biết đến nhiều nhất nhờ căn phòng mang tên ông trong tòa nhà EEMCS 36. Logic Bool được ghi nhận là đã đặt nền móng cho kỷ nguyên thông tin: về cơ bản, là khoa học máy tính. Boole khẳng định rằng: “Không có phương pháp tổng quát nào cho việc giải các câu hỏi trong lý thuyết xác suất có thể được thiết lập mà không nhận biết một cách rõ ràng, không chỉ các cơ sở số học đặc biệt của khoa học, mà còn cả những quy luật phổ quát của tư duy, là nền tảng của mọi lập luận, và dù bản chất của chúng là gì, ít nhất chúng cũng có tính chất toán học về mặt hình thức.”

Nguồn: en.wikipedia.org/wiki/George_Boole.

Đại số của các số bao gồm một số lượng lớn các quy tắc để biến đổi biểu thức. Luật phân phối, ví dụ, nói rằng $x(y + z) = xy + xz$, trong đó x , y và z là các biến đại diện cho bất kỳ số hoặc biểu thức số nào. Luật này có nghĩa là bất cứ khi nào bạn thấy dạng $xy + xz$ trong một biểu thức số, bạn có thể thay thế $x(y + z)$ mà không thay đổi giá trị của biểu thức, và ngược lại. Lưu ý rằng dấu bằng trong $x(y + z) = xy + xz$ có nghĩa “có cùng giá trị, bất kể x , y và z có giá trị số nào”.

Trong đại số Bool, chúng ta làm việc với các giá trị logic thay vì giá trị số. Chỉ có hai giá trị logic, đúng và sai. Chúng ta sẽ viết các giá trị này là $\mathbb{T}$ và $\mathbb{F}$ hoặc 1 và 0. Các ký hiệu $\mathbb{T}$ và $\mathbb{F}$ đóng vai trò tương tự trong đại số Bool như vai trò mà các hằng số như 1 và 3.14159 đóng trong đại số thông thường. Thay vì dấu bằng, đại số Bool sử dụng tương đương logic, $\equiv$, về cơ bản có cùng ý nghĩa.⁵ Ví dụ, với các mệnh đề p , q và r , toán tử $\equiv$ trong $p \wedge (q \wedge r) \equiv (p \wedge q) \wedge r$ có nghĩa “có cùng giá trị, bất kể p , q và r có giá trị logic nào”.

3.1 Cơ sở của Đại số Bool

Nhiều quy tắc của đại số Bool khá hiển nhiên, nếu bạn suy nghĩ một chút về ý nghĩa của chúng. Ngay cả những quy tắc không hiển nhiên cũng có thể được xác minh dễ dàng bằng bảng chân trị. Hình 2 liệt kê các quy tắc quan trọng nhất. Bạn sẽ nhận thấy rằng tất cả các quy tắc này, ngoại trừ quy tắc đầu tiên, đi theo cặp: mỗi quy tắc trong cặp có thể thu được từ quy tắc kia bằng cách hoán đổi $\wedge$ với $\vee$ và $\mathbb{T}$ với $\mathbb{F}$. Điều này giảm số lượng sự kiện bạn phải nhớ.⁶

Làm ví dụ, hãy xác minh quy tắc đầu tiên trong bảng, Luật Phủ định Kép. Luật này

⁵Trong đại số thông thường, dễ bị nhầm lẫn bởi dấu bằng, vì nó có hai vai trò rất khác nhau. Trong một đẳng thức như luật phân phối, nó có nghĩa ‘luôn bằng’. Mặt khác, một phương trình như $x^2 + 3x = 4$ là một phát biểu có thể đúng hoặc sai, tùy thuộc vào giá trị của x . Đại số Bool có hai toán tử, $\equiv$ và $\leftrightarrow$, đóng vai trò tương tự hai vai trò của dấu bằng. $\equiv$ được dùng cho đẳng thức, trong khi $\leftrightarrow$ được dùng trong phương trình có thể đúng hoặc sai.

⁶Đây cũng là một ví dụ của một sự thật tổng quát hơn gọi là *đối ngẫu* (duality), khẳng định rằng cho bất kỳ hằng đúng nào chỉ sử dụng các toán tử $\wedge$, $\vee$ và $\neg$, một hằng đúng khác có thể thu được bằng cách hoán đổi $\wedge$ với $\vee$ và $\mathbb{T}$ với $\mathbb{F}$. Chúng tôi sẽ không cố chứng minh điều này ở đây, nhưng chúng tôi khuyến khích bạn thử!

Luật phủ định kép	$\neg(\neg p) \equiv p$
Luật bài trung (Excluded middle)	$p \vee \neg p \equiv \mathbb{T}$
Luật mâu thuẫn (Contradiction)	$p \wedge \neg p \equiv \mathbb{F}$
Luật đồng nhất (Identity laws)	$\mathbb{T} \wedge p \equiv p$ $\mathbb{F} \vee p \equiv p$
Luật lũy đẳng (Idempotent laws)	$p \wedge p \equiv p$ $p \vee p \equiv p$
Luật giao hoán (Commutative laws)	$p \wedge q \equiv q \wedge p$ $p \vee q \equiv q \vee p$
Luật kết hợp (Associative laws)	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ $(p \vee q) \vee r \equiv p \vee (q \vee r)$
Luật phân phối (Distributive laws)	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
Luật De Morgan (DeMorgan's laws)	$\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$ $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$

Hình 2: Các luật của Đại số Bool. Các luật này đúng với mọi mệnh đề p , q và r .

chỉ là quy tắc ngữ pháp cũ, cơ bản rằng hai lần phủ định thành khẳng định. Mặc dù cách áp dụng quy tắc này vào tiếng Anh còn đáng ngờ—dù nhà ngữ pháp nói gì, “I can’t get no satisfaction” không thực sự có nghĩa “I can get satisfaction”—tính hợp lệ của quy tắc trong logic có thể được xác minh chỉ bằng cách tính hai trường hợp có thể: khi p đúng và khi p sai. Khi p đúng, theo định nghĩa toán tử $\neg$, $\neg p$ sai. Nhưng khi đó, lại theo định nghĩa $\neg$, giá trị của $\neg(\neg p)$ là đúng, bằng giá trị của p . Tương tự, trong trường hợp p sai, $\neg(\neg p)$ cũng sai. Tổ chức thành bảng chân trị, lập luận này có dạng khá đơn giản:

p	$\neg p$	$\neg(\neg p)$
0	1	0
1	0	1

Việc cột đầu và cột cuối giống hệt nhau chứng tỏ sự tương đương logic của p và $\neg(\neg p)$. Điểm ở đây không chỉ là $\neg(\neg p) \equiv p$, mà còn là sự tương đương logic này hợp lệ vì nó có thể được xác minh bằng tính toán dựa trên các định nghĩa liên quan. Tính hợp lệ của nó không suy ra từ việc “nó hiển nhiên” hay “nó là quy tắc ngữ pháp nổi tiếng”.

Info

Sinh viên thường hỏi “Tại sao tôi phải chứng minh điều gì đó khi nó hiển nhiên?” Vấn đề là logic—và toán học nói chung—là thế giới nhỏ của riêng nó với bộ quy tắc riêng. Mặc dù thế giới này có liên hệ nào đó với thế giới thực, khi bạn nói rằng điều gì đó hiển nhiên (trong thế giới thực), bạn không chơi theo quy tắc của thế giới logic. *Phép thuật* thực sự của toán học là bằng cách chơi theo các quy tắc, bạn có thể đưa ra những kết quả rõ ràng *không* hiển nhiên, nhưng vẫn nói lên điều gì đó về thế giới thực hoặc thế giới tính toán—thường là điều thú vị và quan trọng.

Mỗi quy tắc trong Hình 2 có thể được xác minh theo cách tương tự, bằng cách lập bảng chân trị để kiểm tra tất cả các trường hợp có thể.

3.2 Luật thay thế

Điều quan trọng cần hiểu là các biến mệnh đề trong các luật đại số Bool có thể đại diện cho bất kỳ mệnh đề nào, kể cả mệnh đề hợp. Không chỉ đúng, như Luật Phủ định Kép phát biểu, rằng $\neg(\neg p) \equiv p$. Cũng đúng rằng $\neg(\neg q) \equiv q$, rằng $\neg(\neg(p \wedge q)) \equiv (p \wedge q)$, rằng $\neg(\neg(p \rightarrow (q \wedge \neg p))) \equiv (p \rightarrow (q \wedge \neg p))$, và vô số phát biểu khác cùng dạng. Ở đây, ‘phát biểu cùng dạng’ là phát biểu có thể thu được bằng cách thay thế một cái gì đó cho p ở cả hai vị trí mà nó xuất hiện trong $\neg(\neg p) \equiv p$. Làm sao tôi có thể chắc chắn rằng tất cả vô số phát biểu này đều hợp lệ khi tất cả những gì tôi đã kiểm tra chỉ là một bảng chân trị nhỏ hai hàng? Câu trả lời là bất kỳ mệnh đề Q cho trước nào, dù phức tạp đến đâu, cũng có một giá trị chân lý cụ thể, đúng hoặc sai. Vì vậy, câu hỏi về tính hợp lệ của $\neg(\neg Q) \equiv Q$ luôn quy về một trong hai trường hợp tôi đã kiểm tra trong bảng chân trị. (Lưu ý rằng để lập luận này hợp lệ, cùng Q phải được thay thế cho p ở mọi vị trí mà nó xuất hiện.) Mặc dù lập luận này có thể ‘hiển nhiên’, nó không chính xác là một chứng minh, nhưng bây giờ chúng ta sẽ chấp nhận tính hợp lệ của định lý sau:

Định lý 1 (Luật Thay thế Thứ nhất). *Giả sử Q là một mệnh đề bất kỳ, và p là một biến mệnh đề. Xét bất kỳ hằng đúng nào. Nếu (Q) được thay thế cho p ở mọi nơi p xuất hiện trong hằng đúng, thì kết quả cũng là một hằng đúng.*

Vì tương đương logic được định nghĩa qua hằng đúng, cũng đúng rằng khi (Q) được thay thế cho p trong một tương đương logic, kết quả cũng là một tương đương logic.⁷

Luật Thay thế Thứ nhất cho phép bạn làm đại số! Ví dụ, bạn có thể thay thế $p \rightarrow q$ cho p trong luật phủ định kép, $\neg(\neg p) \equiv p$. Điều này cho phép bạn ‘đơn giản hóa’ biểu thức $\neg(\neg(r \rightarrow q))$ thành $r \rightarrow q$ với sự tin tưởng rằng biểu thức kết quả có cùng giá trị logic với biểu thức bạn bắt đầu. (Đó là ý nghĩa của việc $\neg(\neg(r \rightarrow q))$ và $r \rightarrow q$ tương đương logic.) Bạn có thể chơi các thủ thuật tương tự với tất cả các luật trong Hình 2. Thậm chí quan trọng hơn là Luật Thay thế Thứ hai, nói rằng bạn có thể thay thế một biểu thức bằng một biểu thức tương đương logic, bất cứ khi nào nó xuất hiện. Một lần nữa, chúng ta sẽ chấp nhận đây là một định lý mà không cố chứng minh. Thật bất ngờ là khó diễn đạt luật này bằng lời:

Định lý 2 (Luật Thay thế Thứ hai). *Giả sử P và Q là các mệnh đề bất kỳ sao cho $P \equiv Q$. Giả sử R là bất kỳ mệnh đề hợp nào trong đó (P) xuất hiện như một mệnh đề con. Cho R' là mệnh đề thu được bằng cách thay thế (Q) cho lần xuất hiện đó của (P) trong R . Khi đó $R \equiv R'$.*

Lưu ý rằng trong trường hợp này, định lý không yêu cầu (Q) phải được thay thế cho mọi lần xuất hiện của (P) trong R . Bạn được tự do thay thế cho một, hai, hoặc bao nhiêu lần xuất hiện của (P) tùy ý, và kết quả vẫn tương đương logic với R .

Luật Thay thế Thứ hai cho phép chúng ta dùng tương đương logic $\neg(\neg p) \equiv p$ để ‘đơn giản hóa’ biểu thức $q \rightarrow (\neg(\neg p))$ bằng cách thay thế $\neg(\neg p)$ bằng p . Biểu thức kết quả, $q \rightarrow p$, tương đương logic với $q \rightarrow (\neg(\neg p))$ ban đầu. Một lần nữa, chúng ta phải cẩn thận với dấu ngoặc: Sự kiện $p \vee p \equiv p$ không cho phép chúng ta viết lại $q \wedge p \vee p \wedge r$ thành $q \wedge p \wedge r$. Vấn đề là $q \wedge p \vee p \wedge r$ có nghĩa $(q \wedge p) \vee (p \wedge r)$, nên $(p \vee p)$ không phải là biểu

⁷Tôi đã thêm dấu ngoặc quanh Q ở đây vì lý do kỹ thuật. Đôi khi, dấu ngoặc cần thiết để đảm bảo rằng Q được đánh giá như một tổng thể, sao cho giá trị cuối cùng của nó được dùng thay cho p . Ví dụ về điều có thể sai, xét $q \wedge r$. Nếu nó được thay thế theo nghĩa đen cho p trong $\neg(\neg p)$, không có dấu ngoặc, kết quả là $\neg(\neg q \wedge r)$. Nhưng biểu thức này có nghĩa $\neg((\neg q) \wedge r)$, không tương đương với $q \wedge r$. Chúng tôi đã nói hãy luôn viết dấu ngoặc khi nghi ngờ chưa? Xem trang 9.

thức con. Điều này một lần nữa nhấn mạnh tầm quan trọng của việc luôn viết dấu ngoặc trong các công thức mệnh đề.

3.3 Các phép đơn giản hóa

Mảnh ghép cuối cùng của đại số trong đại số Bool là quan sát rằng chúng ta có thể xâu chuỗi các tương đương logic. Nghĩa là, từ $P \equiv Q$ và $Q \equiv R$, suy ra $P \equiv R$. Đây thực sự chỉ là hệ quả của Luật Thay thế Thứ hai. Tương đương $Q \equiv R$ cho phép chúng ta thay thế R cho Q trong phát biểu $P \equiv Q$, cho ra $P \equiv R$. (Nhớ rằng, theo Định nghĩa 2.5, tương đương logic được định nghĩa qua một mệnh đề.) Điều này có nghĩa chúng ta có thể chỉ ra hai mệnh đề hợp tương đương logic bằng cách tìm một chuỗi tương đương logic dẫn từ cái này sang cái kia.

Đây là một ví dụ của chuỗi tương đương logic:

$$\begin{aligned} p \wedge (p \rightarrow q) &\equiv p \wedge (\neg p \vee q) && \text{định nghĩa } p \rightarrow q, \text{ Định lý 2.2} \\ &\equiv (p \wedge \neg p) \vee (p \wedge q) && \text{Luật Phân phối} \\ &\equiv \mathbb{F} \vee (p \wedge q) && \text{Luật Mâu thuẫn, Định lý 2.2} \\ &\equiv (p \wedge q) && \text{Luật Đồng nhất} \end{aligned}$$

Mỗi bước trong chuỗi có lý giải riêng. Trong nhiều trường hợp, luật thay thế được sử dụng mà không nêu rõ. Ở dòng đầu tiên chẳng hạn, định nghĩa $p \rightarrow q$ là $p \rightarrow q \equiv \neg p \vee q$. Luật Thay thế Thứ hai cho phép ta thay thế $(\neg p \vee q)$ cho $(p \rightarrow q)$. Ở dòng cuối, ta ngầm áp dụng Luật Thay thế Thứ nhất cho Luật Đồng nhất, $\mathbb{F} \vee p \equiv p$, để thu được $\mathbb{F} \vee (p \wedge q) \equiv (p \wedge q)$.

Chuỗi tương đương trong ví dụ trên cho phép chúng ta kết luận rằng $p \wedge (p \rightarrow q)$ tương đương logic với $p \wedge q$. Điều này có nghĩa nếu bạn lập bảng chân trị cho hai biểu thức này, giá trị chân lý trong cột $p \wedge (p \rightarrow q)$ sẽ giống hệt với cột $p \wedge q$. Chúng ta biết điều này mà không cần thực sự lập bảng. Không tin ư? Hãy lập bảng chân trị thử. Trong trường hợp này, bảng chỉ có bốn hàng và khá dễ lập. Nhưng đại số Bool có thể được áp dụng trong các trường hợp mà số biến mệnh đề quá lớn khiến bảng chân trị không thực tế.

Note

Hãy làm thêm một ví dụ. Nhớ rằng một mệnh đề hợp là hằng đúng nếu nó đúng với mọi tổ hợp giá trị chân lý có thể của các biến mệnh đề chứa trong nó. Nhưng cách khác để nói cùng một điều là P là hằng đúng nếu $P \equiv \mathbb{T}$. Vì vậy, chúng ta có thể chứng minh một mệnh đề hợp P là hằng đúng bằng cách tìm chuỗi tương đương logic dẫn từ P đến $\mathbb{T}$. Ví dụ:

$$\begin{aligned} ((p \vee q) \wedge \neg p) \rightarrow q &\equiv (\neg((p \vee q) \wedge \neg p)) \vee q && \text{định nghĩa } \rightarrow \\ &\equiv (\neg(p \vee q) \vee \neg(\neg p)) \vee q && \text{Luật De Morgan, Định lý 2.2} \\ &\equiv (\neg(p \vee q) \vee p) \vee q && \text{Phủ định kép, Định lý 2.2} \\ &\equiv (\neg(p \vee q)) \vee (p \vee q) && \text{Luật Kết hợp cho } \vee \\ &\equiv \mathbb{T} && \text{Luật Bài trung} \end{aligned}$$

Từ chuỗi tương đương này, chúng ta kết luận rằng $((p \vee q) \wedge \neg p) \rightarrow q$ là hằng đúng.

Bây giờ, cần thực hành để nhìn một biểu thức và thấy quy tắc nào có thể áp dụng; ví dụ để thấy $(\neg(p \vee q)) \vee (p \vee q)$ là ứng dụng của luật bài trung, bạn cần thay thế $(p \vee q)$ cho p trong luật khi nó được phát biểu trong Hình 2. Thường có nhiều quy tắc có thể áp dụng, và không có hướng dẫn xác định về quy tắc nào bạn nên thử. Đây là điều khiến đại số trở thành một nghệ thuật.

3.4 Thêm các quy tắc của đại số Bool

Chắc chắn không phải tất cả các quy tắc có thể của đại số Bool đều được liệt kê trong Hình 2. Một mặt, có nhiều quy tắc là hệ quả dễ dàng của các quy tắc đã liệt kê. Ví dụ, mặc dù bảng chỉ khẳng định $\mathbb{F} \vee p \equiv p$, cũng đúng rằng $p \vee \mathbb{F} \equiv p$. Điều này có thể kiểm tra trực tiếp hoặc bằng tính toán đơn giản:

$$\begin{aligned} p \vee \mathbb{F} &\equiv \mathbb{F} \vee p \\ &\equiv p \end{aligned}$$

Luật Giao hoán

Luật Đồng nhất như trong bảng

Các quy tắc bổ sung có thể thu được bằng cách áp dụng Luật Giao hoán cho các quy tắc khác trong bảng, và chúng ta sẽ sử dụng các quy tắc như vậy tự do trong tương lai.

Một dạng mở rộng dễ dàng khác có thể áp dụng cho Luật Kết hợp, $(p \vee q) \vee r \equiv p \vee (q \vee r)$. Luật này được phát biểu cho toán tử $\vee$ áp dụng cho ba hạng tử, nhưng nó tổng quát hóa cho bốn hoặc nhiều hạng tử. Ví dụ:

$$\begin{aligned} ((p \vee q) \vee r) \vee s &\equiv (p \vee q) \vee (r \vee s) && \text{theo Luật Kết hợp cho ba hạng tử} \\ &\equiv p \vee (q \vee (r \vee s)) && \text{theo Luật Kết hợp cho ba hạng tử} \end{aligned}$$

Tất nhiên, chúng ta thường viết biểu thức này là $p \vee q \vee r \vee s$, không có dấu ngoặc, vì biết rằng dù đặt ngoặc ở đâu thì giá trị vẫn như nhau.

Note

Một điều nữa bạn nên nhớ là các quy tắc có thể được áp dụng theo *cả hai hướng*. Luật Phân phối, ví dụ, cho phép bạn phân phối p trong $p \vee (q \wedge \neg p)$ để được $(p \vee q) \wedge (p \vee \neg p)$. Nhưng nó cũng có thể được dùng ngược lại để ‘rút gọn’ một hạng tử, ví dụ khi bạn bắt đầu với $(q \vee (p \rightarrow q)) \wedge (q \vee (q \rightarrow p))$ và rút gọn q để được $q \vee ((p \rightarrow q) \wedge (q \rightarrow p))$.

Cho đến nay trong phần này, chúng ta đã làm việc với các luật đại số Bool mà không nói nhiều về ý nghĩa hoặc tại sao chúng hợp lý. Tất nhiên, bạn có thể áp dụng các luật trong tính toán mà không cần hiểu chúng. Nhưng nếu bạn muốn tìm ra *phép tính nào* cần làm, bạn cần hiểu biết. Hầu hết các luật đủ rõ ràng nếu suy nghĩ một chút. Ví dụ, nếu chúng ta đã biết q sai, thì $p \vee q$ sẽ đúng khi p đúng và sai khi p sai. Nghĩa là, $p \vee \mathbb{F}$ có cùng giá trị logic với p . Nhưng đó chính là Luật Đồng nhất cho $\vee$. Một số luật cần thảo luận thêm.

Luật Bài trung, $p \vee \neg p \equiv \mathbb{T}$, nói rằng với bất kỳ mệnh đề p nào, ít nhất một trong p hoặc $\neg p$ phải đúng. Vì $\neg p$ đúng chính xác khi p sai, điều này cũng giống nói rằng p phải đúng hoặc sai. Không có trung gian. Luật Mâu thuẫn, $p \wedge \neg p \equiv \mathbb{F}$, nói rằng không thể cả p và $\neg p$ cùng đúng. Cả hai quy tắc này đều hiển nhiên.

Info

Có những người đặt câu hỏi về luật không có trung gian. Từ những năm 1920, những người như Tarski (người chúng ta sẽ gặp sau) đã nói về các dạng logic khác nơi một giá trị khác biểu diễn ‘không biết’ hoặc ‘chưa được chứng minh’ cũng tồn tại. Bạn cũng có thể thấy điều này trong một số ngôn ngữ lập trình nơi chúng được gọi là ‘boolean ba trạng thái’ (tri-state booleans).

Các logic phi chuẩn này đã được phát triển và cũng dẫn đến những thứ như ‘logic mờ’ (fuzzy logic), mà một số người coi là khá gây tranh cãi. Lotfi Zadeh được ghi nhận là người đầu tiên gọi loại logic này là logic mờ trong công trình về ‘tập mờ’ năm 1965. Zadeh sau đó được trích dẫn nói: “Không sợ bị cuốn vào tranh cãi... Đó cũng là một phần tính cách của tôi. Tôi có thể rất cứng đầu. Điều đó có lẽ có lợi cho sự phát triển của Logic Mờ.”

Nguồn: en.wikipedia.org/wiki/Lotfi_A._Zadeh

Các Luật Phân phối không thể gọi là hiển nhiên, nhưng một vài ví dụ có thể cho thấy chúng hợp lý. Xét phát biểu “Lá bài này là át bích hoặc chuồn”. Rõ ràng, điều này tương đương với “Lá bài này là át bích hoặc lá bài này là át chuồn”. Nhưng đây chính là ví dụ của luật phân phối thứ nhất! Vì, cho a biểu diễn “Lá bài này là ách”, s biểu diễn “Lá bài này là bích” và c biểu diễn “Lá bài này là chuồn”. Khi đó “Lá bài này là át bích hoặc chuồn” có thể dịch sang logic là $a \wedge (s \vee c)$, trong khi “Lá bài này là át bích hoặc lá bài này là át chuồn” trở thành $(a \wedge s) \vee (a \wedge c)$. Và luật phân phối đảm bảo rằng $a \wedge (s \vee c) \equiv (a \wedge s) \vee (a \wedge c)$. Luật phân phối thứ hai cho ta biết, ví dụ, rằng “Lá bài này hoặc là joker hoặc là con mười rô” tương đương logic với “Lá bài này hoặc là joker hoặc con mười, và nó hoặc là joker hoặc là rô”. Nghĩa là, $j \vee (t \wedge d) \equiv (j \vee t) \wedge (j \vee d)$. Các luật phân phối là công cụ mạnh mẽ và bạn nên ghi nhớ chúng bất cứ khi nào đối mặt với hỗn hợp các toán tử $\wedge$ và $\vee$.

Luật De Morgan cũng không thể kém hiển nhiên, vì người ta thường áp dụng sai chúng. May mắn là bạn được thực hành chúng cả trong *Reasoning & Logic* lẫn trong *Computer Organisation*, nên bạn sẽ sớm nắm vững. Quan trọng hơn, chúng cũng có ý nghĩa. Khi xem xét $\neg(p \wedge q)$, bạn nên tự hỏi, làm sao ‘ p và q ’ có thể không đúng. Nó sẽ không đúng nếu p sai *hoặc* nếu q sai (hoặc cả hai). Nghĩa là, $\neg(p \wedge q)$ tương đương với $(\neg p) \vee (\neg q)$. Xét câu “Một con quạ thì lớn và đen.” Nếu một con chim không lớn và đen thì nó không phải quạ. Nhưng chính xác thì ‘không (lớn và đen)’ có nghĩa gì? Làm sao bạn biết khẳng định ‘không (lớn và đen)’ đúng về một thứ gì đó? Điều này sẽ đúng nếu nó hoặc không lớn *hoặc* không đen. (Nó không cần phải là cả hai—nó có thể lớn và trắng, nó có thể nhỏ và đen.) Tương tự, để ‘ p hoặc q ’ không đúng, cả p và q phải sai. Nghĩa là, $\neg(p \vee q)$ tương đương với $(\neg p) \wedge (\neg q)$. Đây là luật De Morgan thứ hai.

Nhớ rằng $p \rightarrow q$ tương đương với $(\neg p) \vee q$, chúng ta có thể áp dụng luật De Morgan để thu được công thức phủ định kép kéo theo:

$$\begin{aligned}\neg(p \rightarrow q) &\equiv \neg((\neg p) \vee q) \\ &\equiv (\neg(\neg p)) \wedge (\neg q) \\ &\equiv p \wedge \neg q\end{aligned}$$

Nghĩa là, $p \rightarrow q$ sai chính xác khi p đúng và q sai. Ví dụ, phủ định của “Nếu bạn có ách, bạn thắng” là “Bạn có ách, và bạn không thắng”. Hãy nghĩ theo cách này: nếu bạn có ách và bạn không thắng, thì phát biểu “Nếu bạn có ách, bạn thắng” là không đúng.

Bài tập

1. Lập bảng chân trị để chứng minh tính đúng đắn của mỗi luật phân phối.
2. Lập các bảng chân trị sau:
 - a) Lập bảng chân trị để chứng minh rằng $\neg(p \wedge q)$ **không** tương đương logic với $(\neg p) \wedge (\neg q)$.
 - b) Lập bảng chân trị để chứng minh rằng $\neg(p \vee q)$ **không** tương đương logic với $(\neg p) \vee (\neg q)$.
 - c) Lập bảng chân trị để chứng minh tính đúng đắn của cả hai Luật De Morgan.
3. Lập bảng chân trị để chứng minh rằng $\neg(p \rightarrow q)$ **không** tương đương logic với bất kỳ biểu thức nào sau đây.
 - a) $(\neg p) \rightarrow (\neg q)$
 - b) $(\neg p) \rightarrow q$
 - c) $p \rightarrow (\neg q)$

Xem lại phần này để tìm công thức thực sự tương đương logic với $\neg(p \rightarrow q)$.

- †4. $\neg(p \leftrightarrow q)$ có tương đương logic với $(\neg p) \leftrightarrow (\neg q)$ không?
5. Trong đại số số, có luật phân phối của phép nhân đối với phép cộng: $x(y + z) = xy + xz$. Luật phân phối của phép cộng đối với phép nhân sẽ trông như thế nào? Nó có phải là luật đúng trong đại số số không?
 6. Các luật phân phối trong Hình 2 đôi khi được gọi là luật phân phối *trái*. Các **luật phân phối phải** nói rằng $(p \vee q) \wedge r \equiv (p \wedge r) \vee (q \wedge r)$ và $(p \wedge q) \vee r \equiv (p \vee r) \wedge (q \vee r)$. Chỉ ra rằng các luật phân phối phải cũng là các luật hợp lệ của đại số Bool. (Lưu ý: Trong thực hành, cả luật phân phối trái và phải đều được gọi đơn giản là luật phân phối, và cả hai có thể được sử dụng tự do trong chứng minh.)
 7. Chỉ ra rằng $p \wedge (q \vee r \vee s) \equiv (p \wedge q) \vee (p \wedge r) \vee (p \wedge s)$ với mọi mệnh đề p, q, r và s . Bằng lời, ta có thể nói rằng phép hội phân phối qua phép tuyển của ba hạng tử. (Nhớ rằng toán tử $\wedge$ được gọi là hội và $\vee$ là tuyển.) Dịch sang logic và xác minh rằng phép hội phân phối qua phép tuyển của bốn hạng tử. Lập luận rằng, thực tế, phép hội phân phối qua phép tuyển của bất kỳ số hạng tử nào.
 8. Có hai luật cơ bản bổ sung của logic, liên quan đến hai biểu thức $p \wedge \mathbb{F}$ và $p \vee \mathbb{T}$. Các luật còn thiếu là gì? Chỉ ra rằng câu trả lời của bạn thực sự là các luật.
 9. Với mỗi cặp mệnh đề sau, chỉ ra rằng hai mệnh đề tương đương logic bằng cách tìm chuỗi tương đương từ cái này sang cái kia. Nêu rõ định nghĩa hoặc luật logic nào biện minh cho mỗi tương đương trong chuỗi.
 - a) $p \wedge (q \wedge p), \quad p \wedge q$
 - b) $(\neg p) \rightarrow q, \quad p \vee q$
 - c) $(p \vee q) \wedge \neg q, \quad p \wedge \neg q$
 - d) $p \rightarrow (q \rightarrow r), \quad (p \wedge q) \rightarrow r$

$$e) (p \rightarrow r) \wedge (q \rightarrow r), \quad (p \vee q) \rightarrow r$$

$$f) p \rightarrow (p \wedge q), \quad p \rightarrow q$$

†10. Với mỗi mệnh đề hợp sau, tìm mệnh đề đơn giản hơn tương đương logic. Cố gắng tìm mệnh đề đơn giản nhất có thể.

$$a) (p \wedge q) \vee \neg q$$

$$b) \neg(p \vee q) \wedge p$$

$$c) p \rightarrow \neg p$$

$$d) \neg p \wedge (p \vee q)$$

$$e) (q \wedge p) \rightarrow q$$

$$f) (p \rightarrow q) \wedge (\neg p \rightarrow q)$$

†11. Diễn đạt phủ định của mỗi câu sau bằng ngôn ngữ tự nhiên:

a) Trời nắng và lạnh.

b) Tôi sẽ ăn stroopwafel hoặc tôi sẽ ăn appeltaart.

c) Nếu hôm nay là thứ Ba, thì đây là nước Bỉ.

d) Nếu bạn đỗ kỳ thi cuối kỳ, bạn đỗ môn học.

12. Áp dụng một trong các luật logic cho mỗi câu sau, và viết lại nó thành câu tương đương. Nêu rõ luật nào bạn đang áp dụng.

a) Tôi sẽ uống cà phê và stroopwafel hoặc appeltaart.

b) Anh ấy không có tài năng cũng không có hoài bão.

c) Bạn có thể ăn oliebollen, hoặc bạn có thể ăn oliebollen.

13. Giả sử đồng thời đúng rằng “Tất cả quả chanh đều vàng” và “Không phải tất cả quả chanh đều vàng”. Suy ra kết luận “Kỳ lân tồn tại”. (Nếu bạn bí, hãy xem en.wikipedia.org/wiki/Principle_of_explosion.)

4 Ứng dụng: Mạch Logic

Như chúng ta đã thấy trong Chương 1, máy tính có tiếng—không phải lúc nào cũng xứng đáng—là ‘logic’. Nhưng về cơ bản, ở sâu bên trong, chúng được tạo từ logic theo nghĩa rất thực. Các khối xây dựng của máy tính là **cổng logic**, là các linh kiện điện tử tính toán giá trị của các mệnh đề đơn giản như $p \wedge q$ và $\neg p$. (Mỗi cổng lại được xây dựng từ các linh kiện điện tử nhỏ hơn gọi là transistor, nhưng điều này không cần quan tâm ở đây: xem môn *Computer Organisation*.)

Info

Đừng lo lắng, mạch logic sẽ được kiểm tra trong *Computer Organisation*, không phải trong *Reasoning & Logic*. Chúng là ví dụ và ứng dụng tốt của logic mệnh đề, và đó là lý do chúng ta nói về chúng trong phần này. Tuy nhiên, các dạng chuẩn (Mục 2.3.4) chắc chắn nằm trong chương trình, vì vậy hãy chú ý!

4.1 Cổng logic

Một dây dẫn trong máy tính có thể ở một trong hai trạng thái, mà chúng ta có thể coi là *bật* (on) và *tắt* (off). Hai trạng thái này có thể được liên kết tự nhiên với các giá trị Bool $\mathbb{T}$ và $\mathbb{F}$. Khi máy tính tính toán, vô số dây dẫn bên trong nó được bật và tắt theo các mẫu được xác định bởi các quy tắc nhất định. Các quy tắc liên quan có thể được biểu đạt tự nhiên nhất bằng logic. Một quy tắc đơn giản có thể là: “bật dây C bất cứ khi nào dây A bật và dây B bật”. Quy tắc này có thể được triển khai trong phần cứng dưới dạng **cổng AND**. Cổng AND là một linh kiện điện tử với hai dây đầu vào và một dây đầu ra, có nhiệm vụ bật đầu ra khi cả hai đầu vào đều bật và tắt đầu ra trong mọi trường hợp khác. Nếu chúng ta liên kết ‘bật’ với $\mathbb{T}$ và ‘tắt’ với $\mathbb{F}$, và đặt tên A và B cho các đầu vào của cổng, thì cổng tính giá trị của biểu thức logic $A \wedge B$. Thực chất, A là một mệnh đề với ý nghĩa “đầu vào thứ nhất bật”, và B là mệnh đề với ý nghĩa “đầu vào thứ hai bật”. Cổng AND hoạt động để đảm bảo đầu ra được mô tả bởi mệnh đề $A \wedge B$. Nghĩa là, đầu ra bật khi và chỉ khi đầu vào thứ nhất bật và đầu vào thứ hai bật.

Như bạn biết từ *Computer Organisation*, **cổng OR** là linh kiện điện tử với hai đầu vào và một đầu ra, bật đầu ra nếu một (hoặc cả hai) đầu vào bật. Nếu các đầu vào được đặt tên A và B , thì cổng OR tính giá trị logic của $A \vee B$. **Cổng NOT** có một đầu vào và một đầu ra, tắt đầu ra khi đầu vào bật và bật đầu ra khi đầu vào tắt. Nếu đầu vào được đặt tên A , thì cổng NOT tính giá trị của $\neg A$.

Cảnh báo

Như chúng tôi đã đề cập, các sách giáo khoa khác có thể dùng ký hiệu khác cho phủ định. Ví dụ dấu gạch ngang trên biến $\bar{x}$ hoặc ký hiệu $\sim$. Trong logic số (và do đó trong môn *Computer Organisation* của bạn), bạn cũng thường thấy ký hiệu $+$ cho ‘hoặc’ và ký hiệu $\cdot$ (chấm) cho ‘và’.

Các ký hiệu chuẩn cho ba cổng logic cơ bản được thể hiện trong Hình 3. Vì các đầu vào và đầu ra của cổng logic chỉ là dây mang tín hiệu bật/tắt, các cổng logic có thể được nối với nhau bằng cách kết nối đầu ra từ một số cổng với đầu vào của các cổng khác. Kết quả là một **mạch logic**.

Cổng AND: ký hiệu hình chữ D với hai đầu vào bên trái và một đầu ra bên phải.

Cổng OR: ký hiệu hình cong với hai đầu vào bên trái và một đầu ra bên phải.

Cổng NOT: ký hiệu hình tam giác với vòng tròn nhỏ ở đầu ra, một đầu vào và một đầu ra.

Hình 3: Các ký hiệu chuẩn cho ba cổng logic cơ bản, và một mạch logic tính giá trị của biểu thức logic $(\neg A) \wedge (B \vee \neg(A \wedge C))$. Các dây đầu vào của mỗi cổng logic ở bên trái, với dây đầu ra ở bên phải. Lưu ý rằng khi các dây cắt nhau trong sơ đồ, chúng thực sự không giao nhau trừ khi có chấm đen tại điểm cắt.

Các loại cổng logic khác tất nhiên cũng có thể. Cổng có thể được tạo để tính $A \rightarrow B$ hoặc $A \oplus B$, chẳng hạn. Tuy nhiên, bất kỳ phép tính nào có thể thực hiện bởi cổng logic đều có thể được thực hiện chỉ bằng cổng AND, OR và NOT, như chúng ta sẽ thấy bên dưới. (Tuy nhiên, trong thực tế, cổng NAND và cổng NOR, tính giá trị $\neg(A \wedge B)$ và $\neg(A \vee B)$ tương ứng, thường được sử dụng vì chúng dễ xây dựng từ transistor hơn cổng AND và OR.)

4.2 Kết hợp cổng để tạo mạch

Ba loại cổng logic được biểu diễn bằng các ký hiệu chuẩn, như thể hiện trong Hình 3. Vì đầu vào và đầu ra của cổng logic chỉ là dây mang tín hiệu bật/tắt, cổng logic có thể được nối với nhau bằng cách kết nối đầu ra từ một số cổng với đầu vào của các cổng khác. Kết quả là một **mạch logic**. Một ví dụ cũng được thể hiện trong Hình 3.

Mạch logic trong hình có ba đầu vào, gắn nhãn A , B và C . Mạch tính giá trị của mệnh đề hợp $(\neg A) \wedge (B \vee \neg(A \wedge C))$. Nghĩa là, khi A biểu diễn mệnh đề “dây đầu vào gắn nhãn A bật”, và tương tự cho B và C , thì đầu ra của mạch bật khi và chỉ khi giá trị của mệnh đề hợp $(\neg A) \wedge (B \vee \neg(A \wedge C))$ đúng.

Cho bất kỳ mệnh đề hợp nào được tạo từ các toán tử $\wedge$, $\vee$ và $\neg$, đều có thể xây dựng mạch logic tính giá trị của mệnh đề đó. Bản thân mệnh đề là bản thiết kế cho mạch. Như đã lưu ý ở Mục 2.1, mọi toán tử logic mà chúng ta đã gặp đều có thể được biểu diễn bằng $\wedge$, $\vee$ và $\neg$, vì vậy thực tế mọi mệnh đề hợp mà chúng ta biết cách viết đều có thể được tính bởi mạch logic.

Cho một mệnh đề được xây dựng từ các toán tử $\wedge$, $\vee$ và $\neg$, việc xây dựng mạch để tính nó khá dễ dàng. Đầu tiên, xác định toán tử chính trong mệnh đề—toán tử có giá trị được tính cuối cùng. Xét $(A \vee B) \wedge \neg(A \wedge B)$. Mạch này có hai giá trị đầu vào, A và B , được biểu diễn bởi các dây vào mạch. Mạch có một dây đầu ra biểu diễn giá trị đã tính của mệnh đề. Toán tử chính trong $(A \vee B) \wedge \neg(A \wedge B)$ là $\wedge$ đầu tiên, tính giá trị của biểu thức tổng thể bằng cách kết hợp giá trị của các biểu thức con $A \vee B$ và $\neg(A \wedge B)$. Toán tử $\wedge$ này tương ứng với cổng AND trong mạch tính đầu ra cuối cùng.

Một khi toán tử chính đã được xác định và biểu diễn dưới dạng cổng logic, bạn chỉ cần xây dựng mạch để tính đầu vào cho toán tử đó. Trong ví dụ, đầu vào cho cổng AND chính đến từ hai mạch con. Một mạch con tính giá trị $A \vee B$ và mạch con kia tính giá trị $\neg(A \wedge B)$. Xây dựng mỗi mạch con là bài toán riêng, nhưng nhỏ hơn bài toán ban đầu. Cuối cùng, bạn sẽ đến cổng có đầu vào trực tiếp từ một trong các dây đầu vào— A hoặc B trong trường hợp này—thay vì từ mạch con.

4.3 Từ mạch đến mệnh đề

Vậy, mọi mệnh đề hợp đều được tính bởi mạch logic với một dây đầu ra. Điều ngược lại có đúng không? Nghĩa là, cho một mạch logic với một đầu ra, liệu có mệnh đề nào biểu diễn giá trị đầu ra theo giá trị đầu vào không? Không hẳn. Khi bạn nối các cổng logic để tạo mạch, không có gì ngăn bạn tạo ra **vòng phản hồi** (feedback loop). Vòng phản hồi xảy ra khi đầu ra của một cổng được kết nối—có thể qua một hoặc nhiều cổng trung gian—trở lại đầu vào của cùng cổng đó. Vòng phản hồi không thể được mô tả bằng mệnh đề hợp, cơ bản vì không có nơi để bắt đầu, không có đầu vào để liên kết với biến mệnh đề. Nhưng vòng phản hồi là điều duy nhất có thể sai. Mạch logic không chứa vòng phản hồi nào được gọi là **mạch logic tổ hợp** (combinatorial logic circuit). Mọi mạch logic tổ hợp với đúng một đầu ra tính giá trị của một mệnh đề hợp nào đó. Các biến mệnh đề trong mệnh đề hợp chỉ là tên gắn với các dây đầu vào của mạch. (Tất nhiên, nếu mạch có nhiều đầu ra, bạn chỉ cần dùng một mệnh đề khác cho mỗi đầu ra.)

Chìa khóa để hiểu tại sao điều này đúng là lưu ý rằng mỗi dây trong mạch—không chỉ dây đầu ra cuối cùng—biểu diễn giá trị của một mệnh đề nào đó. Hơn nữa, một khi bạn biết mệnh đề nào được biểu diễn bởi mỗi dây đầu vào của một cổng, hiển nhiên mệnh đề nào được biểu diễn bởi đầu ra: Bạn chỉ cần kết hợp các mệnh đề đầu vào với toán tử $\wedge$, $\vee$ hoặc $\neg$ tương ứng, tùy thuộc loại cổng. Để tìm mệnh đề gắn với đầu ra cuối cùng, bạn

chỉ cần bắt đầu từ đầu vào và di chuyển qua mạch, gắn nhãn dây đầu ra của mỗi cổng với mệnh đề mà nó biểu diễn.

4.4 Dạng chuẩn tắc tuyển

Các mệnh đề hợp tương ứng tự nhiên với mạch logic tổ hợp. Nhưng chúng ta vẫn chưa giải quyết hoàn toàn câu hỏi về mức độ mạnh mẽ của các mạch và mệnh đề này. Chúng ta đã xem xét một số toán tử logic và lưu ý rằng tất cả đều có thể biểu diễn bằng $\wedge$, $\vee$ và $\neg$. Nhưng liệu có thể có các toán tử khác không thể biểu diễn như vậy? Tương đương, liệu có thể có các loại cổng logic khác—có thể với số lượng lớn đầu vào—mà phép tính của chúng không thể tái tạo bằng cổng AND, OR và NOT? Bất kỳ toán tử logic hay cổng logic nào cũng tính giá trị cho mỗi tổ hợp giá trị logic có thể của đầu vào. Chúng ta luôn có thể lập bảng chân trị cho thấy đầu ra cho mỗi tổ hợp đầu vào có thể. Hóa ra, cho bất kỳ bảng chân trị nào, đều có thể tìm một mệnh đề chỉ chứa toán tử $\wedge$, $\vee$ và $\neg$, mà giá trị cho mỗi tổ hợp đầu vào chính xác bằng bảng đó.

Để thấy tại sao điều này đúng, cần giới thiệu một loại mệnh đề hợp cụ thể. Định nghĩa **hạng tử đơn giản** (simple term) là biến mệnh đề hoặc phủ định của biến mệnh đề. Một **hội của các hạng tử đơn giản** gồm một hoặc nhiều hạng tử đơn giản nối bằng toán tử $\wedge$. (“Hội của một hạng tử đơn giản” chỉ là một hạng tử đơn giản đứng một mình. Điều này có thể không hợp ngữ pháp, nhưng đó là cách các nhà toán học nghĩ.) Một số ví dụ về hội của các hạng tử đơn giản là $p \wedge q$, p , $\neg q$ và $p \wedge \neg r \wedge \neg w \wedge s \wedge t$. Cuối cùng, chúng ta có thể lấy một hoặc nhiều hội như vậy và nối chúng thành ‘tuyển của các hội của các hạng tử đơn giản’. Đây là loại mệnh đề hợp chúng ta cần. Chúng ta có thể tránh sự dư thừa bằng cách giả định rằng không biến mệnh đề nào xuất hiện nhiều hơn một lần trong một hội (vì $p \wedge p$ có thể thay bằng p , và nếu cả p và $\neg p$ cùng xuất hiện trong một hội, thì giá trị hội đó sai, và nó có thể được loại bỏ). Chúng ta cũng có thể giả định rằng cùng hội không xuất hiện hai lần trong tuyển.

Cảnh báo

Các dạng chuẩn là một phần chương trình của *Reasoning & Logic*. Các dạng chuẩn này, như Dạng Chuẩn tắc Tuyển (mục này) và Dạng Chuẩn tắc Hội (xem bài tập), rất quan trọng trong logic mệnh đề. Cũng có các dạng chuẩn cho các logic khác, chẳng hạn cho *logic vị từ* mà chúng ta sẽ xem xét ở Mục 2.4 tiếp theo.

Định nghĩa 6. Một mệnh đề hợp được gọi là ở **dạng chuẩn tắc tuyển** (Disjunctive Normal Form – DNF) nếu nó là tuyển của các hội của các hạng tử đơn giản, và hơn nữa, mỗi biến mệnh đề xuất hiện nhiều nhất một lần trong mỗi hội và mỗi hội xuất hiện nhiều nhất một lần trong tuyển.

Sử dụng p, q, r, s, A và B làm biến mệnh đề, đây là một vài ví dụ về mệnh đề ở dạng chuẩn tắc tuyển:

$$\begin{aligned} & (p \wedge q \wedge r) \vee (p \wedge \neg q \wedge r \wedge s) \vee (\neg p \wedge \neg q) \\ & (p \wedge \neg q) \\ & (A \wedge \neg B) \vee (\neg A \wedge B) \\ & p \vee (\neg p \wedge q) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge \neg q \wedge \neg r \wedge w) \end{aligned}$$

Các mệnh đề ở DNF chính xác là thứ chúng ta cần để xử lý các bảng đầu vào/đầu ra mà chúng ta đã thảo luận. Bất kỳ bảng nào cũng có thể được tính bởi một mệnh đề ở

dạng chuẩn tắc tuyển. Từ đó suy ra rằng có thể xây dựng mạch để tính bảng đó chỉ bằng cổng AND, OR và NOT.

Định lý 3. Xét bảng liệt kê giá trị đầu ra logic cho mỗi tổ hợp giá trị của một số biến mệnh đề. Giả sử ít nhất một giá trị đầu ra là đúng. Khi đó tồn tại mệnh đề chứa các biến đó sao cho giá trị của mệnh đề cho mỗi tổ hợp giá trị có thể của các biến chính xác là giá trị được chỉ định trong bảng. Có thể chọn mệnh đề ở dạng chuẩn tắc tuyển.

Chứng minh. Xét bất kỳ hàng nào trong bảng mà giá trị đầu ra là T. Tạo hội các hạng tử đơn giản như sau: Với mỗi biến p có giá trị T ở hàng đó, thêm p vào hội; với mỗi biến q có giá trị F ở hàng đó, thêm $\neg q$ vào hội. Giá trị của hội này là T cho tổ hợp giá trị biến ở hàng đó, vì mỗi hạng tử trong hội đều đúng cho tổ hợp biến đó. Hơn nữa, với bất kỳ tổ hợp giá trị biến nào khác, giá trị hội sẽ là F, vì ít nhất một hạng tử đơn giản trong hội sẽ sai.

Lấy tuyển của tất cả các hội được xây dựng theo cách này, cho mỗi hàng trong bảng có giá trị đầu ra là đúng. Tuyển này có giá trị T khi và chỉ khi một trong các hội tạo thành nó có giá trị T—và điều đó chính xác khi giá trị đầu ra trong bảng là T. Vì vậy, tuyển các hội này thỏa mãn yêu cầu của định lý. $\square$

Info

Đây là chứng minh đầu tiên của một khẳng định không tầm thường mà chúng ta đã thấy. Bạn sẽ học về định lý và chứng minh, và các kỹ thuật chứng minh, ở cuối chương này và trong Chương 3.

Làm ví dụ, xét bảng trong Hình 4. Bảng này chỉ định giá trị đầu ra mong muốn cho mỗi tổ hợp giá trị có thể của các biến mệnh đề p , q và r . Nhìn vào hàng thứ hai của bảng, nơi giá trị đầu ra là đúng. Theo chứng minh định lý, hàng này tương ứng với hội $(\neg p \wedge \neg q \wedge r)$. Hội này đúng khi p sai, q sai và r đúng; trong mọi trường hợp khác nó sai, vì trong bất kỳ trường hợp khác nào, ít nhất một trong các hạng tử $\neg p$, $\neg q$ hoặc r sẽ sai. Hai hàng khác có đầu ra đúng cho hai hội nữa. Ba hội được kết hợp để tạo mệnh đề DNF $(\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)$. Mệnh đề này tính tất cả giá trị đầu ra được chỉ định trong bảng. Sử dụng mệnh đề này làm bản thiết kế, chúng ta có mạch logic có đầu ra khớp với bảng.

p	q	r	đầu ra	
F	F	F	F	
F	F	T	T	$(\neg p \wedge \neg q \wedge r)$
F	T	F	F	
F	T	T	T	$(\neg p \wedge q \wedge r)$
T	F	F	F	
T	F	T	F	
T	T	F	F	
T	T	T	T	$p \wedge q \wedge r$

Hình 4: Bảng đầu vào/đầu ra chỉ định đầu ra mong muốn cho mỗi tổ hợp giá trị của các biến mệnh đề p , q và r . Mỗi hàng có đầu ra T tương ứng với một hội, được thể hiện bên cạnh hàng đó. Tuyển các hội này là mệnh đề có giá trị đầu ra chính xác là giá trị trong bảng.

Bây giờ, cho bất kỳ mạch logic tổ hợp nào, có nhiều mạch khác có cùng hành vi đầu vào/đầu ra. Khi hai mạch có cùng bảng đầu vào/đầu ra, các mệnh đề hợp gắn với hai mạch tương đương logic. Nói cách khác, các mệnh đề tương đương logic tạo ra mạch có cùng hành vi đầu vào/đầu ra. Trong thực tế, chúng ta thường ưu tiên mạch đơn giản hơn. Sự tương ứng giữa mạch và mệnh đề cho phép chúng ta áp dụng đại số Bool cho việc đơn giản hóa mạch.

Note

Sự ưu tiên cho đơn giản hơn áp dụng cho mệnh đề hợp, dù chúng có tương ứng với mạch hay không. Chúng ta thường ưu tiên dạng tương đương của mệnh đề mà đơn giản hơn. Bất kỳ mệnh đề nào cũng có mệnh đề tương đương ở DNF. Vì vậy khi chứng minh định lý về mệnh đề hợp, chỉ cần xét các mệnh đề DNF. Điều này có thể làm chứng minh dễ viết hơn.

Ví dụ, xét mệnh đề DNF tương ứng với bảng trong Hình 4. Trong $(\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)$, ta có thể rút gọn $(q \wedge r)$ từ hai hạng tử cuối, cho $(\neg p \wedge \neg q \wedge r) \vee ((\neg p \vee p) \wedge (q \wedge r))$. Vì $\neg p \vee p \equiv \mathbb{T}$, và $\mathbb{T} \wedge Q \equiv Q$ với mọi mệnh đề Q , điều này đơn giản thành $(\neg p \wedge \neg q \wedge r) \vee (q \wedge r)$. Lại áp dụng luật phân phối để rút gọn r , cho $((\neg p \wedge \neg q) \vee q) \wedge r$. Mệnh đề hợp này tương đương logic với mệnh đề ban đầu, nhưng triển khai trong mạch chỉ cần năm cổng logic, thay vì mười cổng cần cho mệnh đề ban đầu.⁸

Nếu bạn bắt đầu với mạch thay vì mệnh đề, thường có thể tìm mệnh đề gắn kết, đơn giản hóa nó bằng đại số Bool, và dùng mệnh đề đơn giản để xây mạch tương đương đơn giản hơn mạch ban đầu. Và đơn giản mệnh đề về DNF thường là cách tiếp cận hợp lý.

Note

Một cách đơn giản mệnh đề là dùng **bản đồ Karnaugh** (Karnaugh-map hay K-map) như bạn sẽ học trong *Computer Organisation*. Dùng K-map bạn có thể tìm cái gọi là ‘tổng tối thiểu của tích’ (minimal sum of products). Lưu ý rằng tổng của tích chỉ là mệnh đề viết ở DNF. Trong môn *Reasoning & Logic* chúng tôi có thể yêu cầu bạn chuyển mệnh đề sang dạng DNF. Bạn có thể chọn dùng quy tắc viết lại hoặc K-map tùy ý.

4.5 Cộng nhị phân

Tất cả những điều này giải thích mối quan hệ giữa logic và mạch, nhưng không giải thích tại sao mạch logic nên được dùng trong máy tính. Một phần giải thích nằm ở việc máy tính sử dụng số nhị phân. Số nhị phân là chuỗi các số không và một. Số nhị phân dễ biểu diễn trong thiết bị điện tử như máy tính: Mỗi vị trí trong số tương ứng với một dây. Khi dây bật, nó biểu diễn một; khi dây tắt, nó biểu diễn không. Khi chúng ta nghĩ về logic, các trạng thái dây tương tự biểu diễn đúng và sai, nhưng cả hai biểu diễn chỉ là diễn giải của thực tế, đó là dây bật hoặc tắt. Câu hỏi là liệu diễn giải có hiệu quả không.

Khi dây được coi là biểu diễn số không và một, chúng ta có thể xây mạch tính toán với số nhị phân. Phép tính nào? Bất kỳ phép tính nào chúng ta muốn! Nếu biết đáp án

⁸Không, chúng tôi không đếm sai. Có mười một toán tử logic trong biểu thức ban đầu, nhưng bạn chỉ cần mười cổng trong mạch: Dùng một cổng NOT duy nhất để tính $\neg p$, và kết nối đầu ra cổng đó với hai cổng AND khác nhau. Tái sử dụng đầu ra cổng logic là cách hiển nhiên để đơn giản mạch mà không tương ứng với phép toán nào trên mệnh đề.

cho mỗi tổ hợp đầu vào, thì theo Định lý 2.3 ta có thể xây mạch tính đáp án đó. Tất nhiên, quy trình mô tả trong định lý chỉ thực tế cho mạch nhỏ, nhưng mạch nhỏ có thể dùng làm khối xây dựng cho tất cả mạch tính toán trong máy tính.

Ví dụ, xét phép cộng nhị phân. Để cộng hai số thập phân thông thường, bạn xếp chúng chồng lên nhau và cộng các chữ số trong mỗi cột. Trong mỗi cột, cũng có thể có nhớ từ cột trước. Để cộng một cột, bạn chỉ cần nhớ một số lượng nhỏ quy tắc, như $7 + 6 + 1 = 14$ và $3 + 5 + 0 = 8$. Với phép cộng nhị phân, còn dễ hơn, vì các chữ số chỉ là 0 và 1. Chỉ có tám quy tắc:

$$\begin{array}{ll} 0 + 0 + 0 = 00 & 1 + 0 + 0 = 01 \\ 0 + 0 + 1 = 01 & 1 + 0 + 1 = 10 \\ 0 + 1 + 0 = 01 & 1 + 1 + 0 = 10 \\ 0 + 1 + 1 = 10 & 1 + 1 + 1 = 11 \end{array}$$

Ở đây, chúng ta viết mỗi tổng bằng hai chữ số. Trong phép cộng nhiều cột, một trong các chữ số này được nhớ sang cột tiếp theo. Như vậy, chúng ta có phép tính với ba đầu vào và hai đầu ra. Chúng ta có thể lập bảng đầu vào/đầu ra cho mỗi đầu ra. Các bảng được thể hiện trong Hình 5. Chúng ta biết các bảng này có thể triển khai dưới dạng mạch tổ hợp, vì vậy chúng ta biết mạch có thể cộng số nhị phân. Để cộng số nhị phân nhiều chữ số, chỉ cần một bản sao mạch cộng cơ bản cho mỗi cột trong tổng.

A	B	C	đầu ra	A	B	C	đầu ra
0	0	0	0	0	0	0	0
0	0	1	1	0	0	1	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	1	1
1	0	0	1	1	0	0	0
1	0	1	0	1	0	1	1
1	1	0	0	1	1	0	1
1	1	1	1	1	1	1	1

Hình 5: Bảng đầu vào/đầu ra cho phép cộng ba chữ số nhị phân A , B và C .

Bài tập

- Chỉ dùng cổng AND, OR và NOT, vẽ mạch tính giá trị của mỗi mệnh đề $A \rightarrow B$, $A \leftrightarrow B$ và $A \oplus B$.
- Với mỗi mệnh đề sau, tìm mạch logic tổ hợp tính mệnh đề đó:
 - $A \wedge (B \vee \neg C)$
 - $(p \wedge q) \wedge \neg(p \wedge \neg q)$
 - $(p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$
 - $\neg(A \wedge (B \vee C)) \vee (B \wedge \neg A)$
- Tìm mệnh đề hợp được tính bởi mỗi mạch cho trước. (Xem sách gốc để biết hình các mạch.)

4. Phần này mô tả phương pháp tìm mệnh đề hợp được tính bởi bất kỳ mạch logic tổ hợp nào. Phương pháp này thất bại nếu bạn cố áp dụng cho mạch chứa vòng phản hồi. Điều gì sai? Cho ví dụ.
5. Chỉ ra rằng mọi mệnh đề hợp không phải mâu thuẫn đều tương đương với mệnh đề ở dạng chuẩn tắc tuyển. (Lưu ý: Ta có thể bỏ hạn chế mệnh đề không phải mâu thuẫn bằng cách chấp nhận rằng ‘ $\mathbb{F}$ ’ được tính là mệnh đề ở dạng chuẩn tắc tuyển. $\mathbb{F}$ tương đương logic với mọi mâu thuẫn.)
6. Mệnh đề ở **dạng chuẩn tắc hội** (Conjunctive Normal Form – CNF) là hội của các tuyển của các hạng tử đơn giản (với quy ước, giống như định nghĩa DNF, rằng một hạng tử đơn cũng tính là hội hoặc tuyển). Chỉ ra rằng mọi mệnh đề hợp không phải hằng đúng đều tương đương logic với mệnh đề ở dạng chuẩn tắc hội. (Gợi ý: Điều gì xảy ra nếu bạn lấy phủ định của mệnh đề DNF và áp dụng Luật De Morgan?)
7. Dùng các luật đại số Bool để đơn giản hóa mỗi mạch sau. (Xem sách gốc để biết hình các mạch.)
8. Thiết kế mạch triển khai các bảng đầu vào/đầu ra cho phép cộng, như trong Hình 5. Cố gắng làm mạch đơn giản nhất có thể. (Các mạch dùng trong máy tính thực cho mục đích này đơn giản hơn những gì bạn có thể nghĩ ra, nhưng cách tiếp cận tổng quát dùng logic để thiết kế mạch máy tính là đúng đắn. Nếu bạn muốn tìm hiểu thêm, môn *Digital Systems* năm thứ hai mô tả thiết kế mạch chi tiết hơn.)